Image & Sequential Data

2026 Winter DL Session 5주차 Part 2

고려대학교 KOREA UNIVERSITY

KUBIG DATA SCIENCE & AI

목차

KUBIG 2026 Winter DL Session

04
Sequential Data

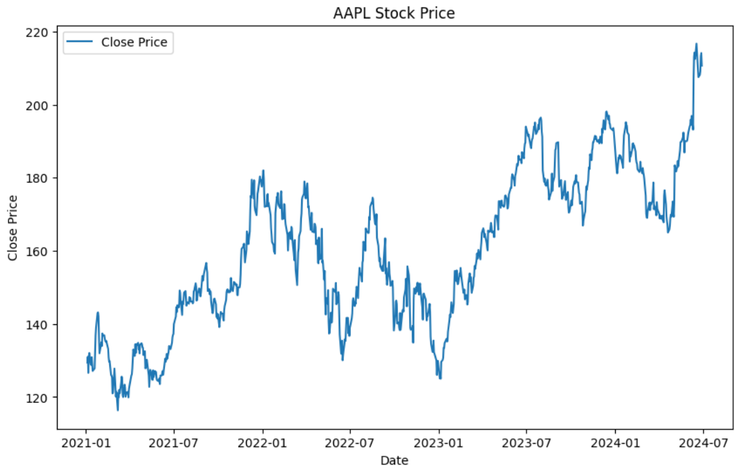
04. Sequential Data

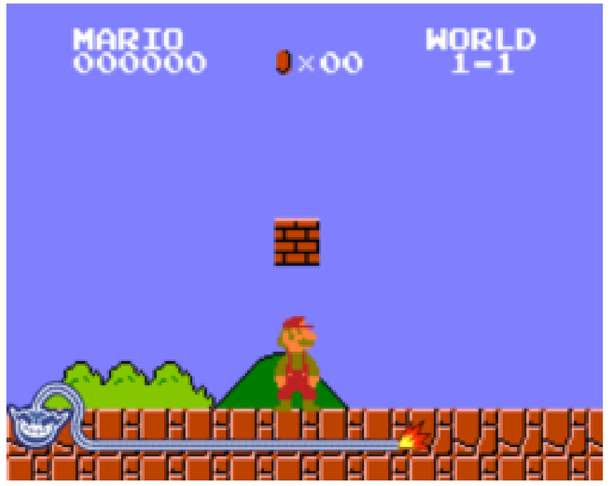
Principles for Sequential Data

- Sequential Data

: 데이터 집합 내의 객체들이 특정 순서를 따르는 데이터. 그 순서가 변경될 경우 고유의 특성이 변질됨.

  - Time series data
  - Text data
  - Sequential decision making





04. Sequential Data

Principles for Sequential Data

- Sequential pattern 처리가 어려운 이유

: Sequence length 는 고정적이지 않음.

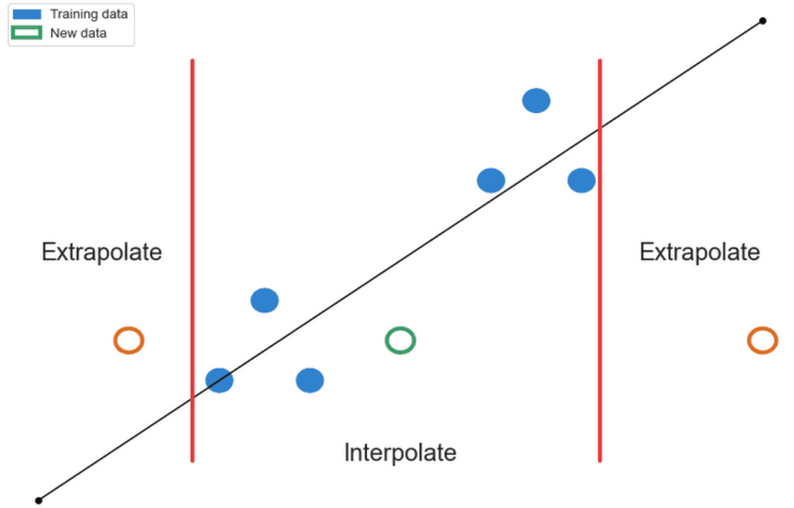
  - variable length → MLP(x), CNN(o)

:Extrapolation (외삽) vs Interpolation (보간)

  - 패턴 분석이 예측보다 쉬움. (ex. stock price)

: Sequential data are non-i.i.d.

  - sequence 순서는 치명적.
  - 순서가 바뀔 경우 의미/특징이 완전히 달라짐. e.g. dog bites man vs man bites dog



04. Sequential Data

Principles for Sequential Data

→ Sequentiality

input data는 sequential하게 처리되어야 한다. → 데이터의 순서를 섞으면 안 된다.

$$\mathbb{P}(X_1, \dots, X_n) = \prod_{k=1}^{n} \mathbb{P}(X_k | X_1, \dots, X_{k-1})$$

P(man bites dog) ≠ P(dog bites man)

$$\mathbb{P}(X_1, X_2, X_3) \neq \mathbb{P}(X_1)\mathbb{P}(X_2)\mathbb{P}(X_3)$$

04. Sequential Data

Principles for Sequential Data

→ Sequentiality

input data는 sequential하게 처리되어야 한다. → 데이터의 순서를 섞으면 안 된다.

$$\mathbb{P}(X_1, \dots, X_n) = \prod_{k=1}^{n} \mathbb{P}(X_k | X_1, \dots, X_{k-1})$$

P(man bites dog) ≠ P(dog bites man)

$$\mathbb{P}(X_1, X_2, X_3) \neq \mathbb{P}(X_1)\mathbb{P}(X_2)\mathbb{P}(X_3)$$

→ Temporal Invariance

translation invariance in sequence order

$$\mathbb{P}(X_{1+j}, \dots, X_{n+j}) = \prod_{k=1}^{n} \mathbb{P}(X_k | X_1, \dots, X_{k-1})$$

$j = 1,2,3, \dots,$

04. Sequential Data

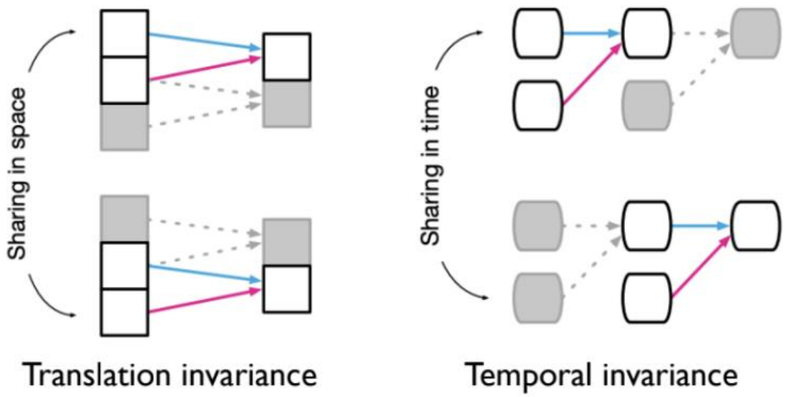
Principles for Sequential Data

→ Temporal Invariance

translation invariance in sequence order

$$\mathbb{P}(X_{1+j}, \dots, X_{n+j}) = \prod_{k=1}^{n} \mathbb{P}(X_k | X_1, \dots, X_{k-1})$$

$j = 1,2,3, \dots,$



$j = 1,2,3, \dots,$

$$= \prod_{k=1}^{n} \mathbb{P}(X_{k+j} | X_{1+j} = \mathbf{x}_1, \dots, X_{k-1+j} = \mathbf{x}_{k-1})$$

$$= f_\theta(\mathbf{x}_1, \dots, \mathbf{x}_{k-1})$$

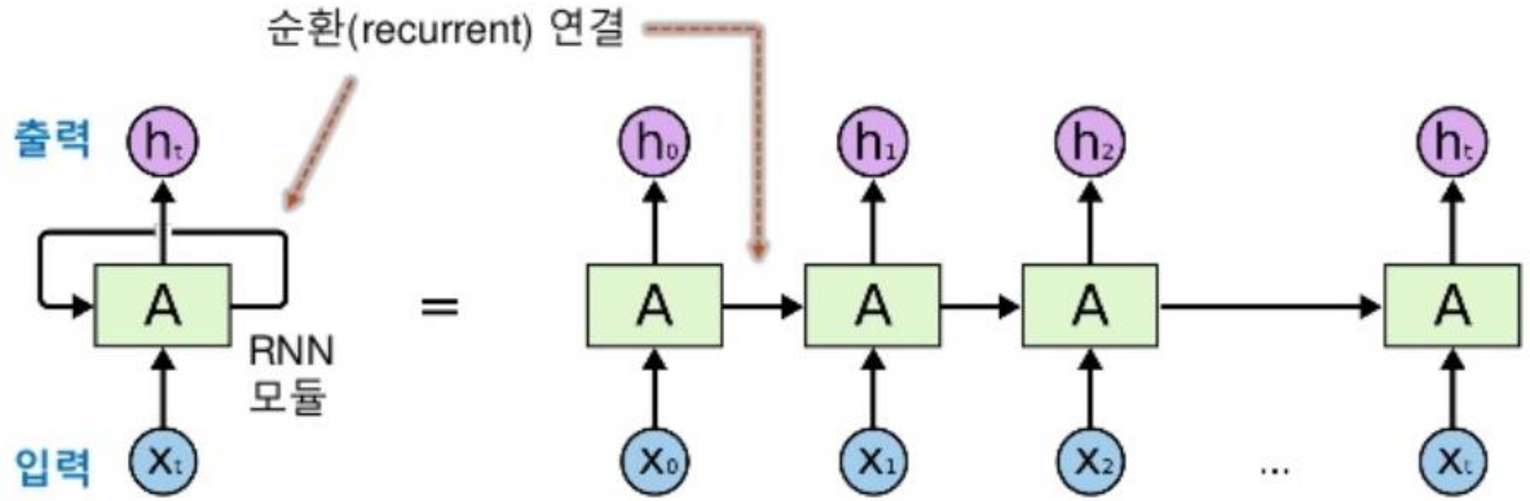
shared parameter

** Parameter sharing (가중치 공유)는
invariance를 설계하기 위한 핵심 방법입니다!

04. Sequential Data

Principles for Sequential Data

- Recurrent Neural Network (RNN)의 핵심 기능
  - Sequentiality: input data를 order에 따라 처리
  - Temporal Invariance: 동일한 정보는 시점에 무관하게 동일하게 처리
  - Context Dependency: 이전 정보를 기억해서 반영
  - Variable Length: 샘플 길이에 따른 가변성



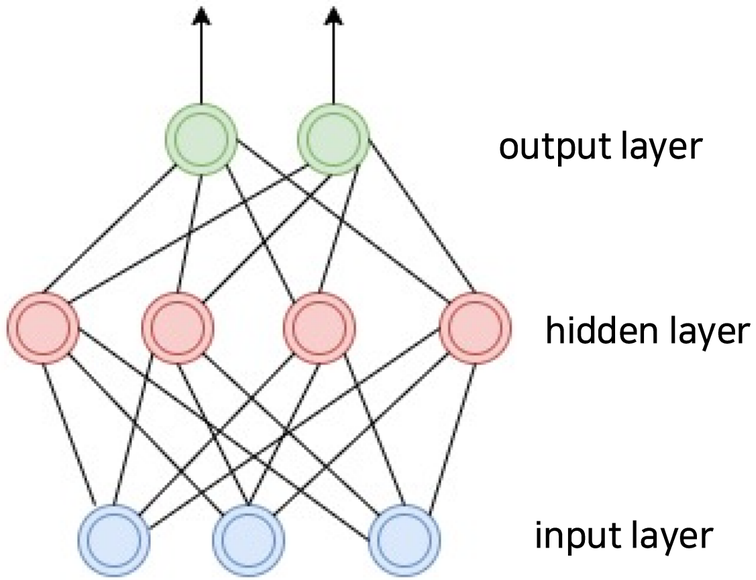
05

RNN & LSTM

05. RNN & LSTM

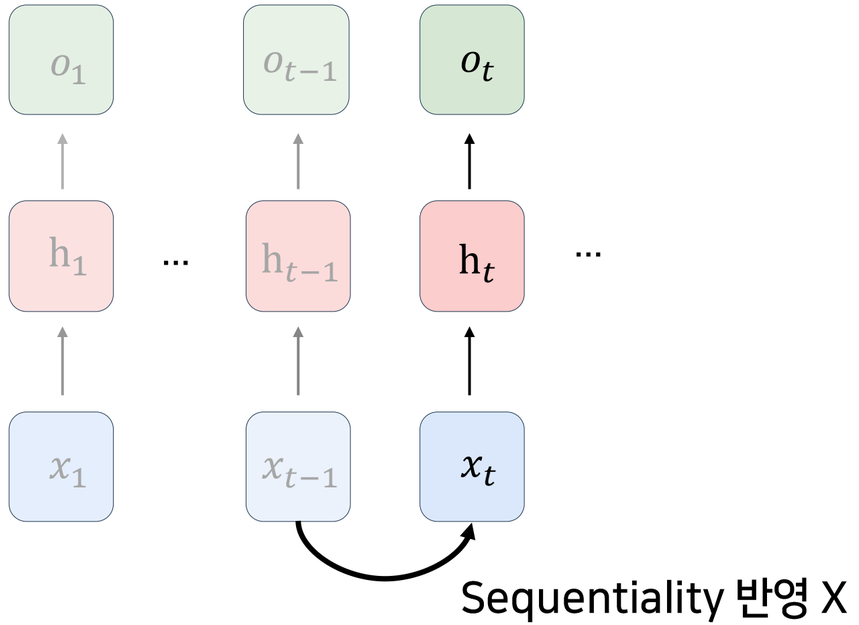
인공 신경망 (DNN) vs 순환 신경망 (RNN)

- 인공 신경망 (DNN)



$$\mathbf{h}_t = f(W_{\mathbf{h}x}\boldsymbol{x}_t)$$
$$\boldsymbol{o}_t = g(W_{o\mathbf{h}}\mathbf{h}_t)$$

DNN



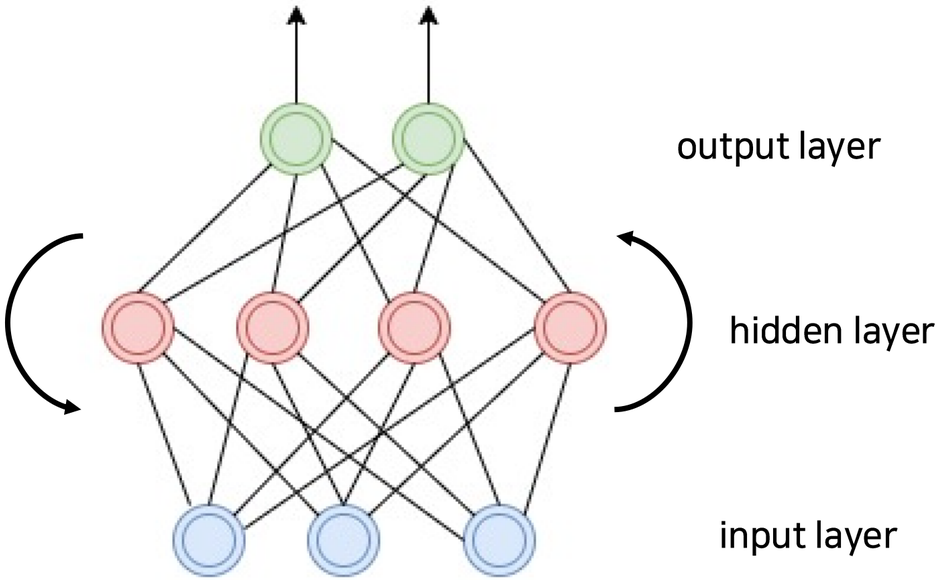
DNN

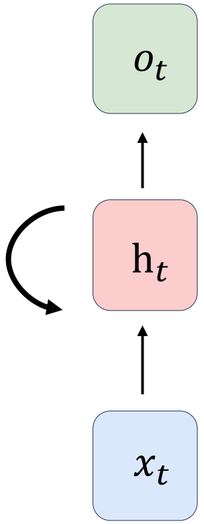
05. RNN & LSTM

인공 신경망 (DNN) vs 순환 신경망 (RNN)

- 순환 신경망 (RNN)

: 이전 시점 정보들을 반영하여 시계열 데이터 모델링에 적합한 인공 신경망 모델



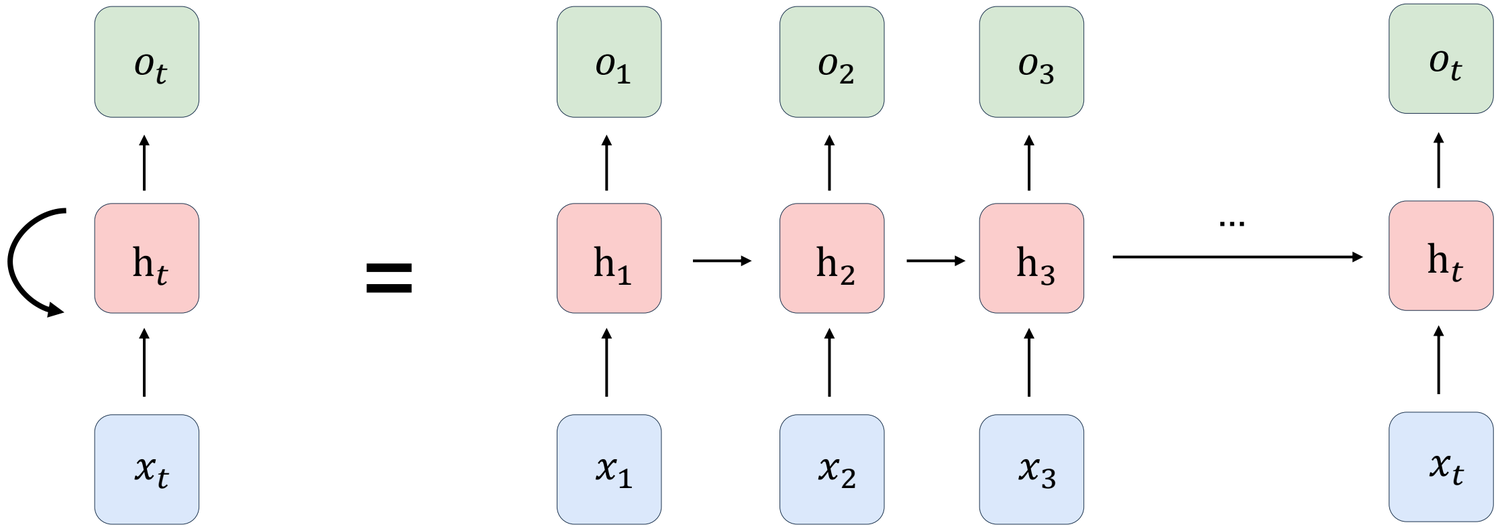


RNN: hidden state를 output layer 방향 + 다시 hidden layer로도 전달

05. RNN & LSTM

순환 신경망 (RNN)

$$\mathbf{h}_t = f(W_{\mathbf{h}x}x_t) + \text{이전 시점들에 대한 정보}$$
$$o_t = g(W_{o\mathbf{h}}\mathbf{h}_t)$$



RNN: hidden state를 output layer 방향 + 다시 hidden layer로도 전달

05. RNN & LSTM

순환 신경망 (RNN)

- 직관적 예시

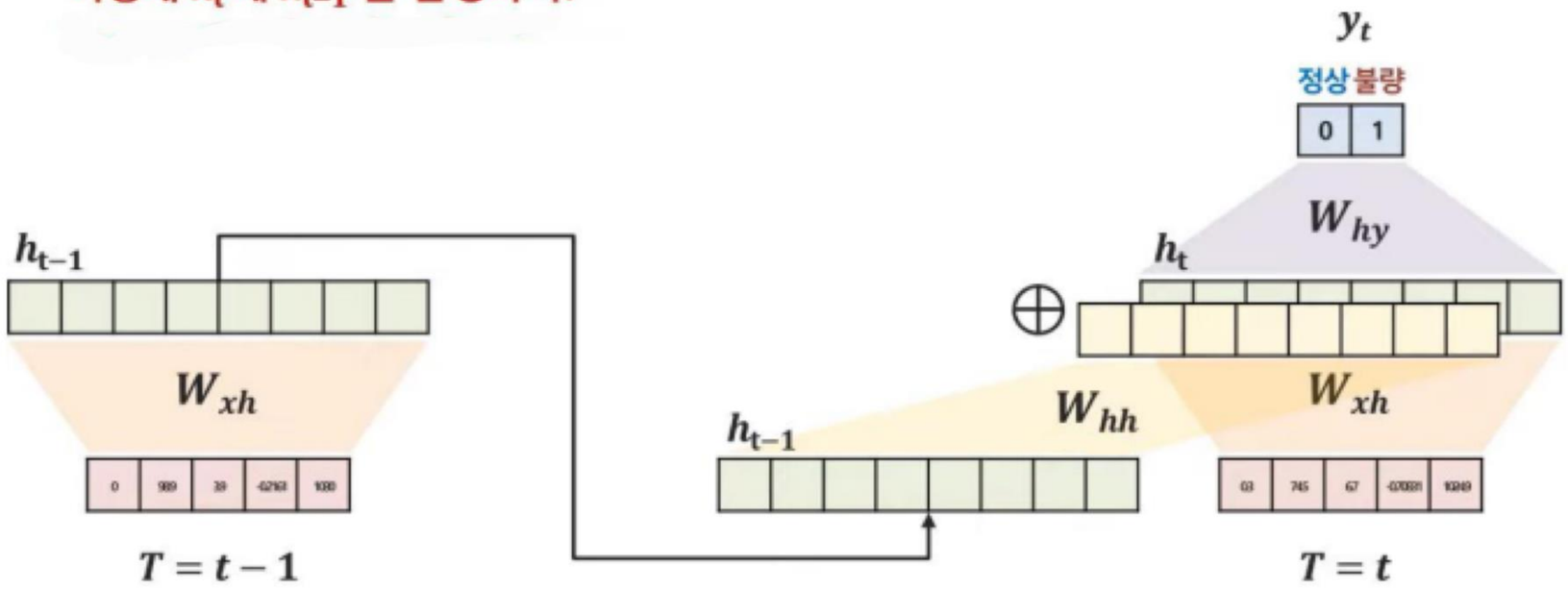
	시간	센서1	센서2	센서3	센서4	센서5	상태
$t-2$	12:00	0	98.9	39	0.19	0.21	정상
$t-1$	13:00	0	98.9	39	0.21	0.23	정상
t	14:00	0.3	74.5	67	0.23	0.24	불량

$$h_t = f(W_{xh}x_t + W_{hh}h_{t-1})$$

$$y_t = g(W_{hy}h_t)$$

$$f(\cdot) = tanh\,,\ g(\cdot) = softmax$$

"어떻게 h_t에 h_{t-1} 를 합성하지?"



05. RNN & LSTM

순환 신경망 (RNN)

- 직관적 예시

	시간	센서1	센서2	센서3	센서4	센서5	상태
$t-2$	12:00	0	98.9	3.9	0.19	0.21	정상
$t-1$	13:00	0	98.9	3.9	0.21	0.23	정상
t	14:00	0.3	74.5	6.7	0.23	0.24	불량

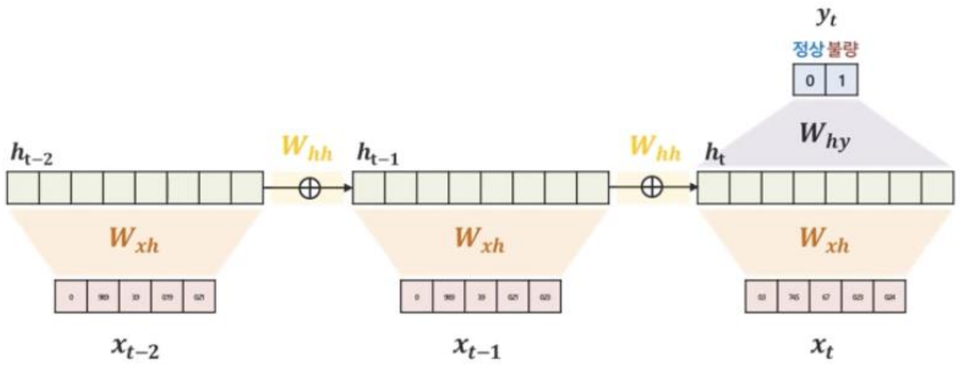
$$h_{t-1} = f(W_{xh}x_{t-1} + W_{hh}h_{t-2})$$

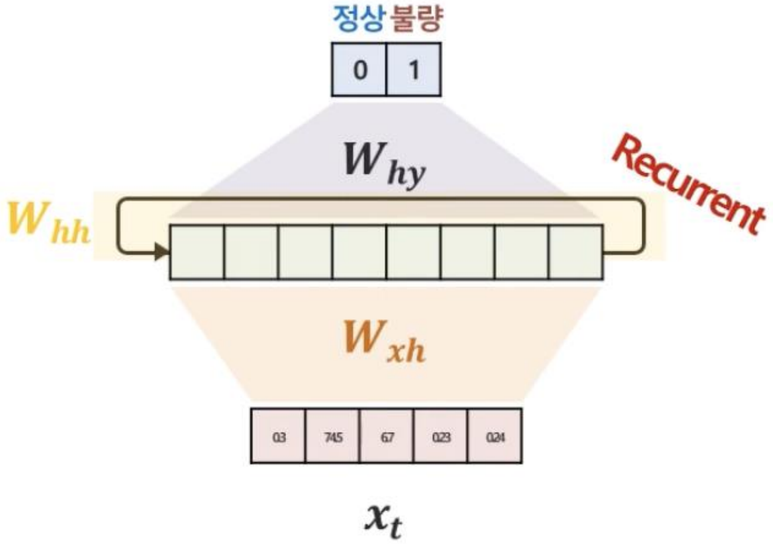
$$h_t = f(W_{xh}x_t + W_{hh}h_{t-1})$$

$$y_t = g(W_{hy}h_t)$$

$$f(\cdot) = tanh\ ,\ g(\cdot) = softmax$$

"RNN은 이전 정보들이 순환(반복)하여 입력"





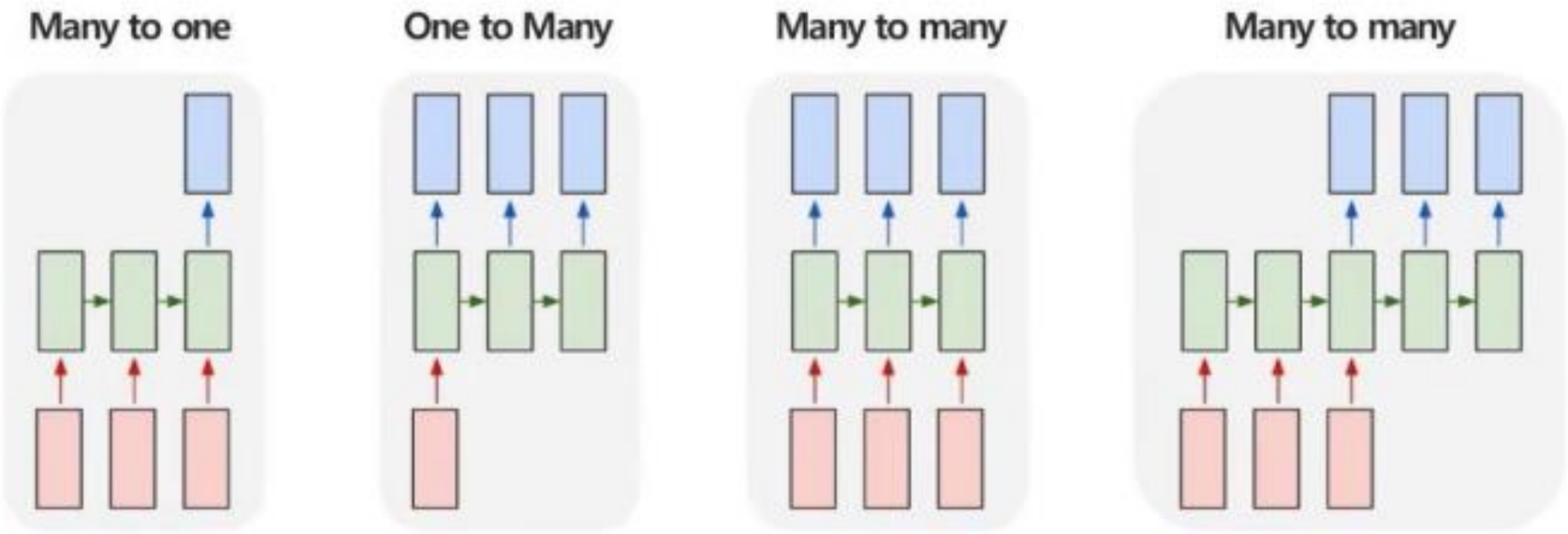
05. RNN & LSTM

순환 신경망 (RNN)

- RNN 다양한 구조

: 순차적으로 입력하고 순차적으로 예측하는 것이 가능
: 입력의 길이, 예측의 길이에 따라 다음과 같이 구분됨

필요에 따라 다양하고
유연하게 구조를 만들 수 있음



<감정분석>
단어들 → 감성점수

사진 설명 붙이기
사진 → 단어들

<번역>
단어들 → 단어들

05. RNN & LSTM

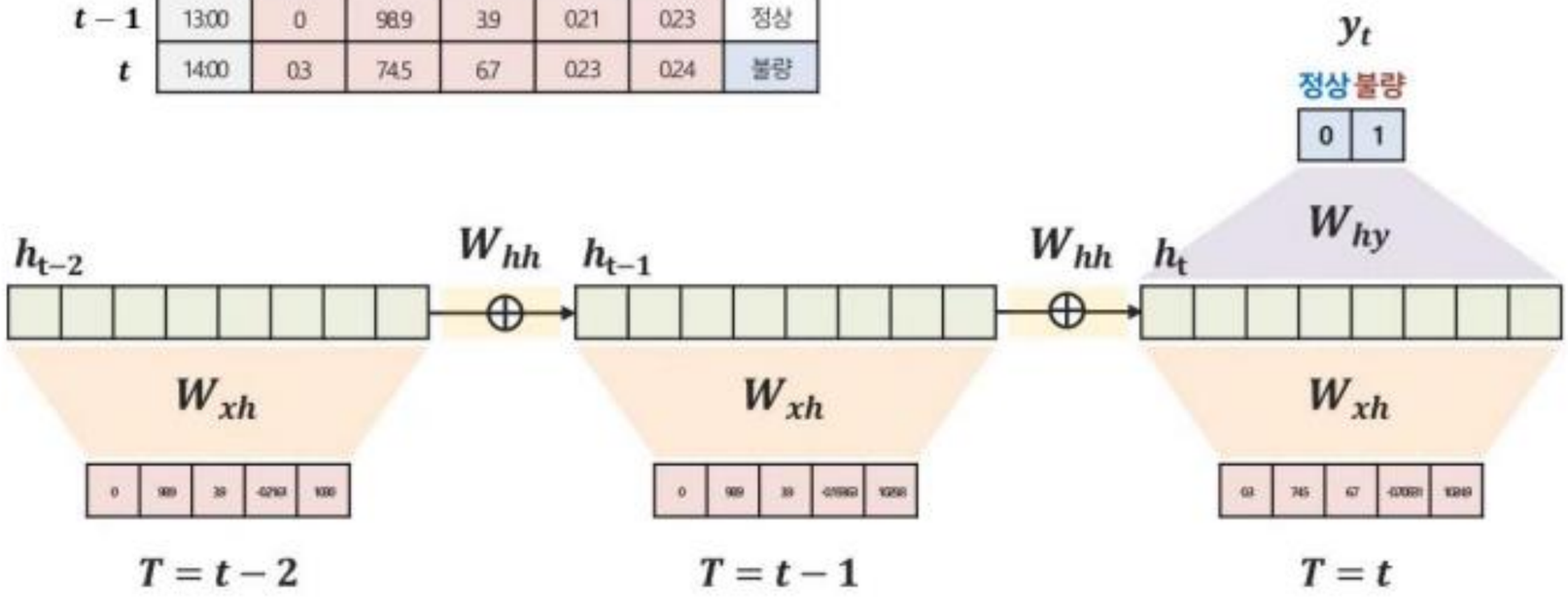
순환 신경망 (RNN)

- 1) Many to one 구조

: 여러 시점 X로 하나의 Y를 예측하는 문제

: ex. 여러 시점의 다변량 센서 데이터 → 특정 시점의 제품 상태 예측

	시간	센서1	센서2	센서3	센서4	센서5	상태
$t-2$	12:00	0	98.9	39	0.19	0.21	정상
$t-1$	13:00	0	98.9	39	0.21	0.23	정상
t	14:00	0.3	74.5	67	0.23	0.24	불량

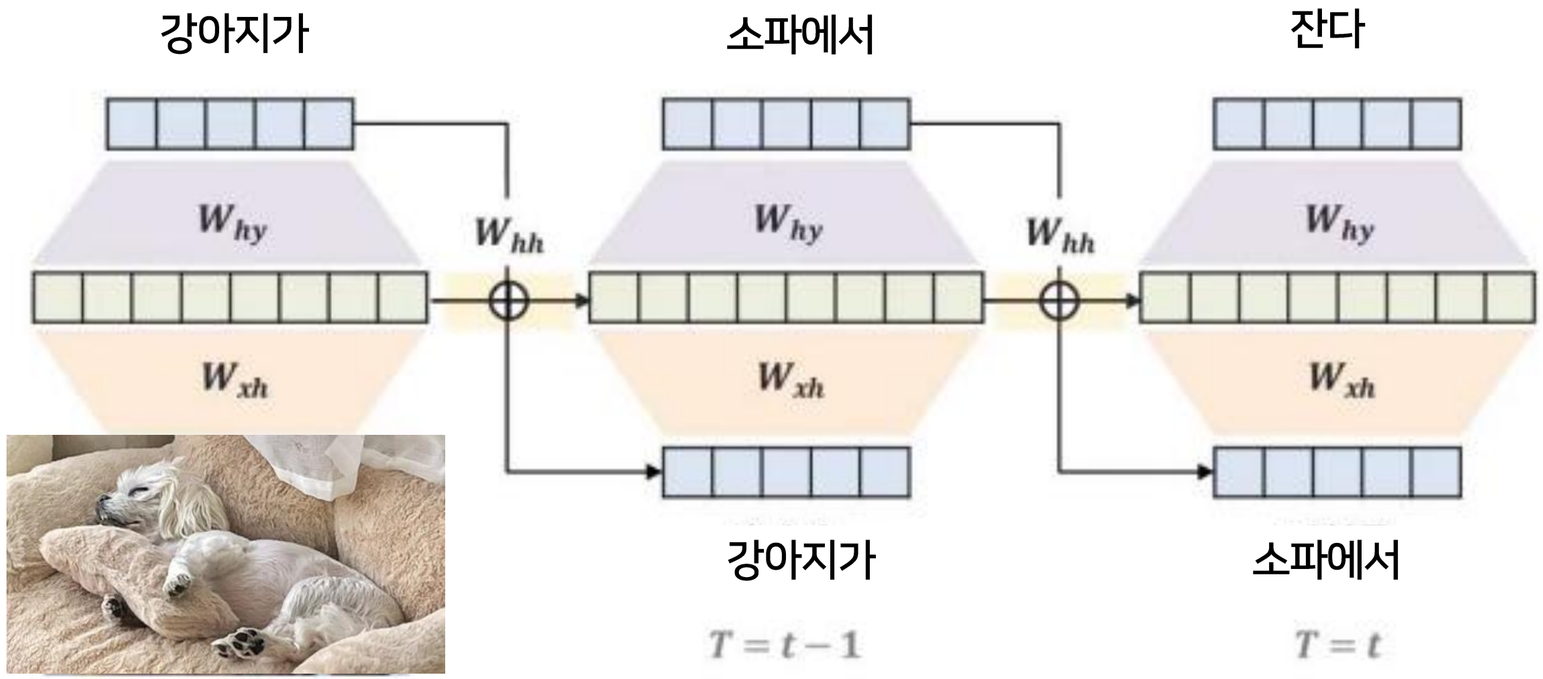


05. RNN & LSTM

순환 신경망 (RNN)

- 2) One to many 구조

: 단일 시점 X로 순차적인 Y를 예측하는 문제
: ex. 이미지 캡셔닝 (이미지 한 장 → 이미지에 대한 정보를 글 생성)

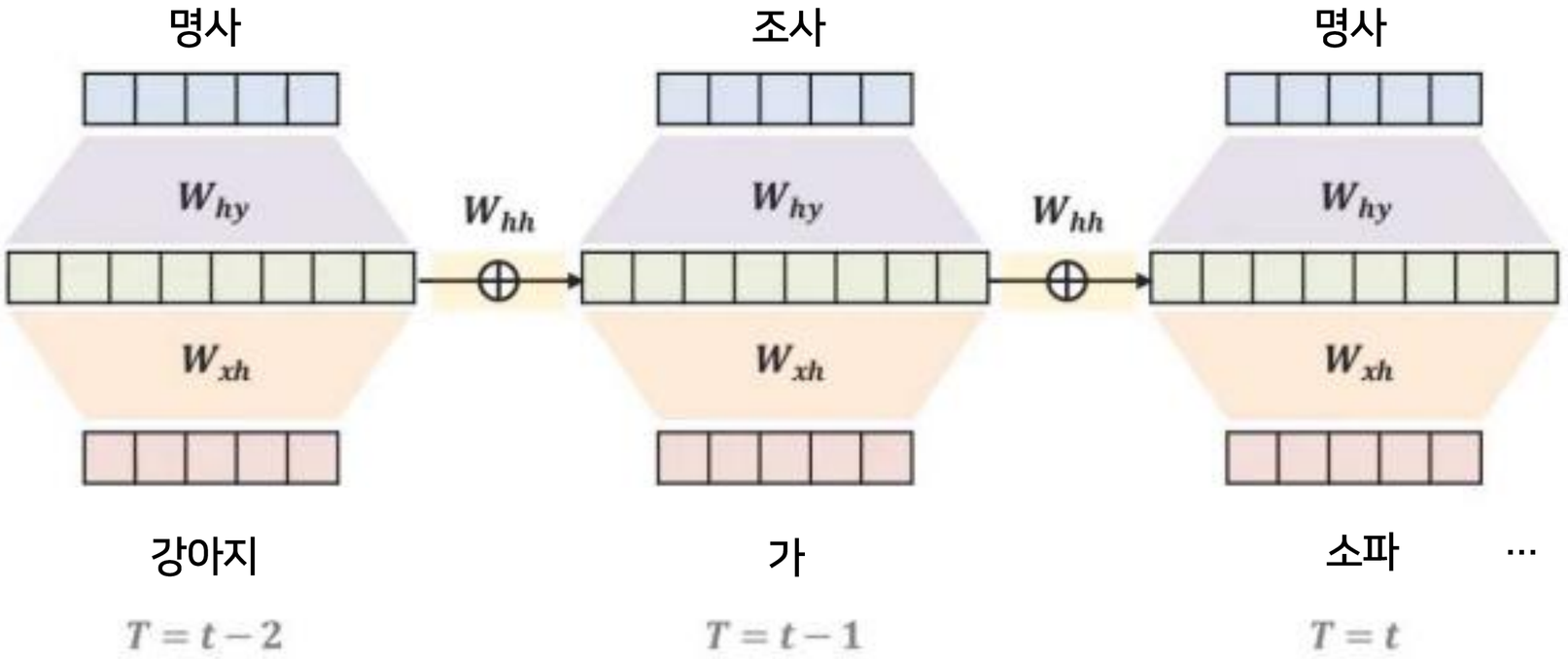


05. RNN & LSTM

순환 신경망 (RNN)

- 3-1) Many to many 구조

: 순차적인 X로 순차적인 Y를 예측하는 문제
: ex. POS Tagging (문장 → 각 단어의 품사 예측)

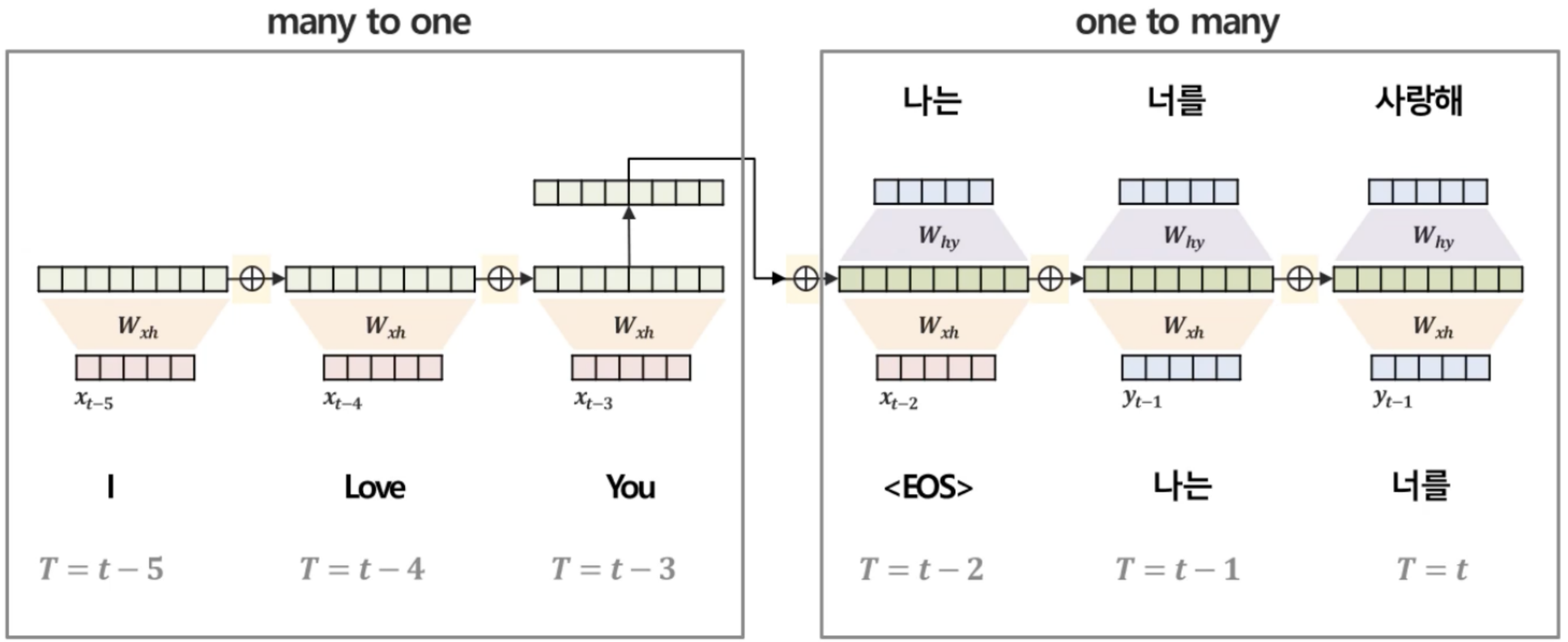


05. RNN & LSTM

순환 신경망 (RNN)

- 3-2) Many to many 구조 (many to one + one to many)

: 순차적인 X로 순차적인 Y를 예측하는 문제
: ex. 번역 (영어 문장 → 한글 문장)



05. RNN & LSTM

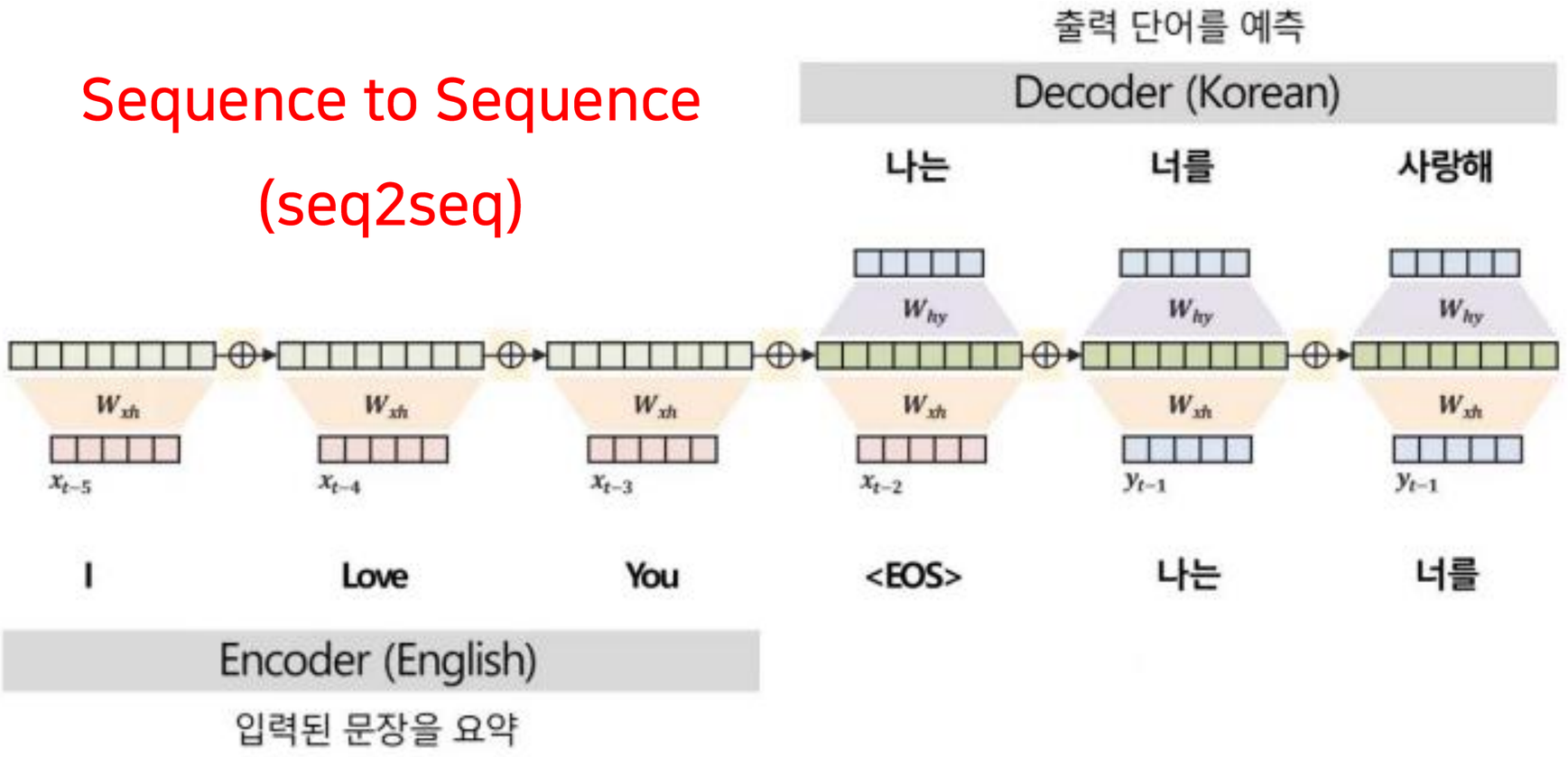
순환 신경망 (RNN)

- 3-2) Many to many 구조 (many to one + one to many)

: 순차적인 X로 순차적인 Y를 예측하는 문제
: ex. 번역 (영어 문장 → 한글 문장)

Sequence to Sequence

(seq2seq)



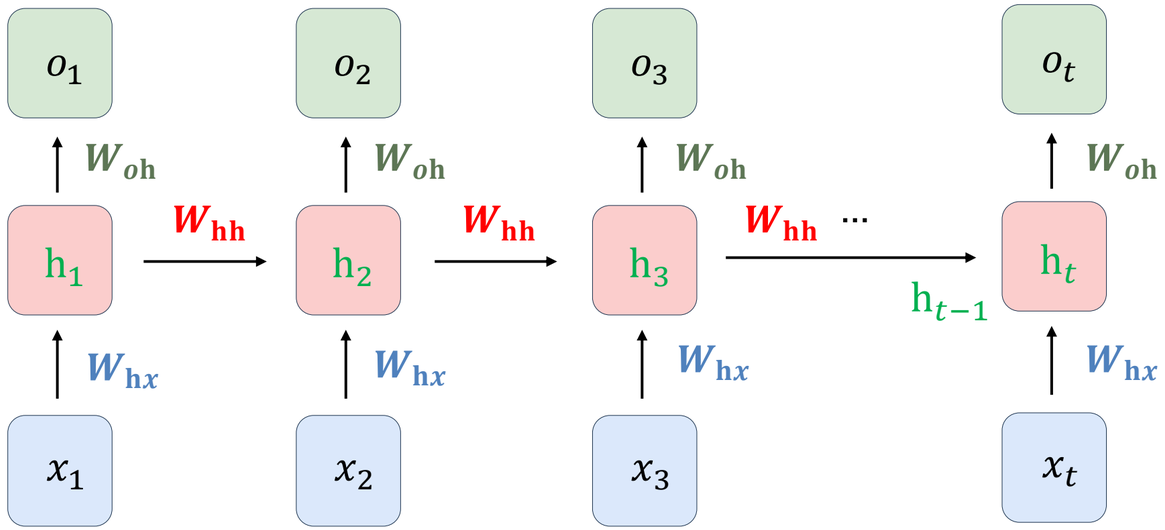
05. RNN & LSTM

RNN의 학습

: 가중치 행렬 $\boldsymbol{W}_{\mathrm{h}x}, \boldsymbol{W}_{\mathrm{hh}}, \boldsymbol{W}_{o\mathrm{h}}$가 매 time step 마다 동일 (weight sharing)
: 업데이트 해야 할 가중치(파라미터)가 input 길이에 상관 없이 고정
: time step 이 거듭될 때마다 context information이 누적

$\boldsymbol{W}_{\mathrm{h}x}: D_{\mathrm{h}} \times d$
$\boldsymbol{W}_{\mathrm{hh}}: D_{\mathrm{h}} \times D_{\mathrm{h}}$
$\boldsymbol{W}_{o\mathrm{h}}: d \times D_{\mathrm{h}}$

$x_t: d \times 1$
$\boldsymbol{W}_{\mathrm{hh}}: D_{\mathrm{h}} \times D_{\mathrm{h}}$
$\boldsymbol{W}_{o\mathrm{h}}: d \times D_{\mathrm{h}}$

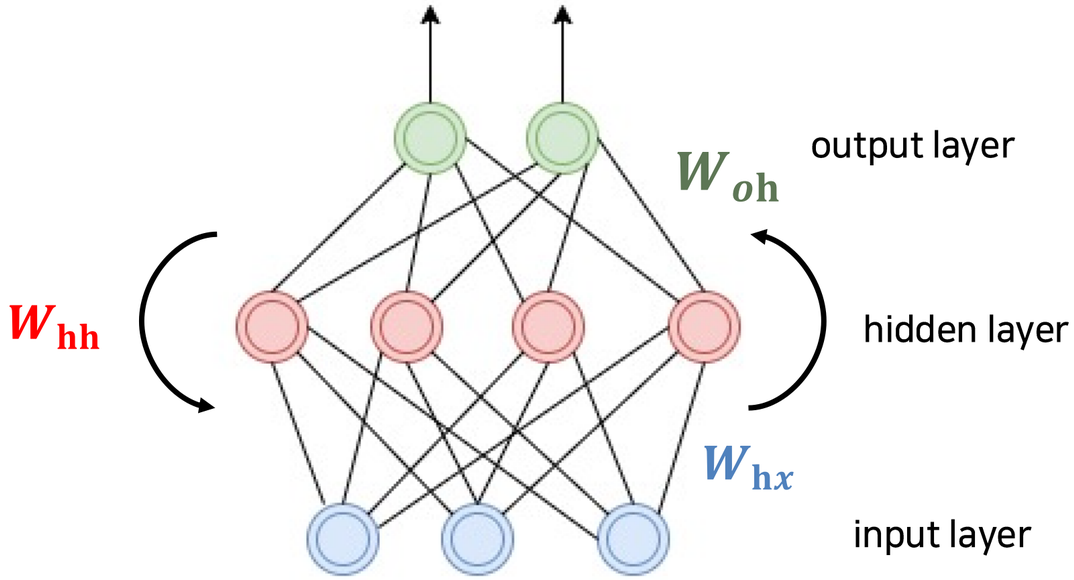


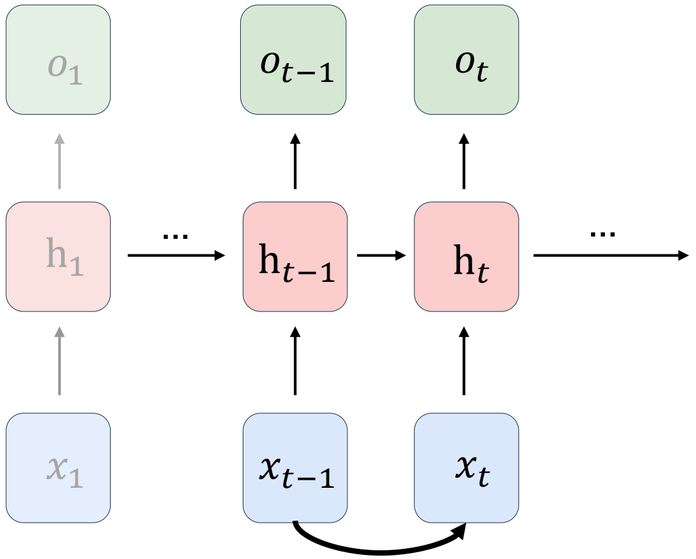
05. RNN & LSTM

RNN의 학습 (Forward pass)

$$\mathbf{h}_{t-1} = f(W_{\mathrm{h}x}\boldsymbol{x}_{t-1} + W_{\mathrm{hh}}\mathbf{h}_{t-2})$$
$$\mathbf{h}_{t} = f(W_{\mathrm{h}x}\boldsymbol{x}_{t} + W_{\mathrm{hh}}\mathbf{h}_{t-1})$$
$$\hat{\boldsymbol{o}}_{t} = g(W_{o\mathrm{h}}\boldsymbol{h}_{t})$$

$$f(\cdot) = \tanh, g(\cdot) = \mathrm{softmax}$$

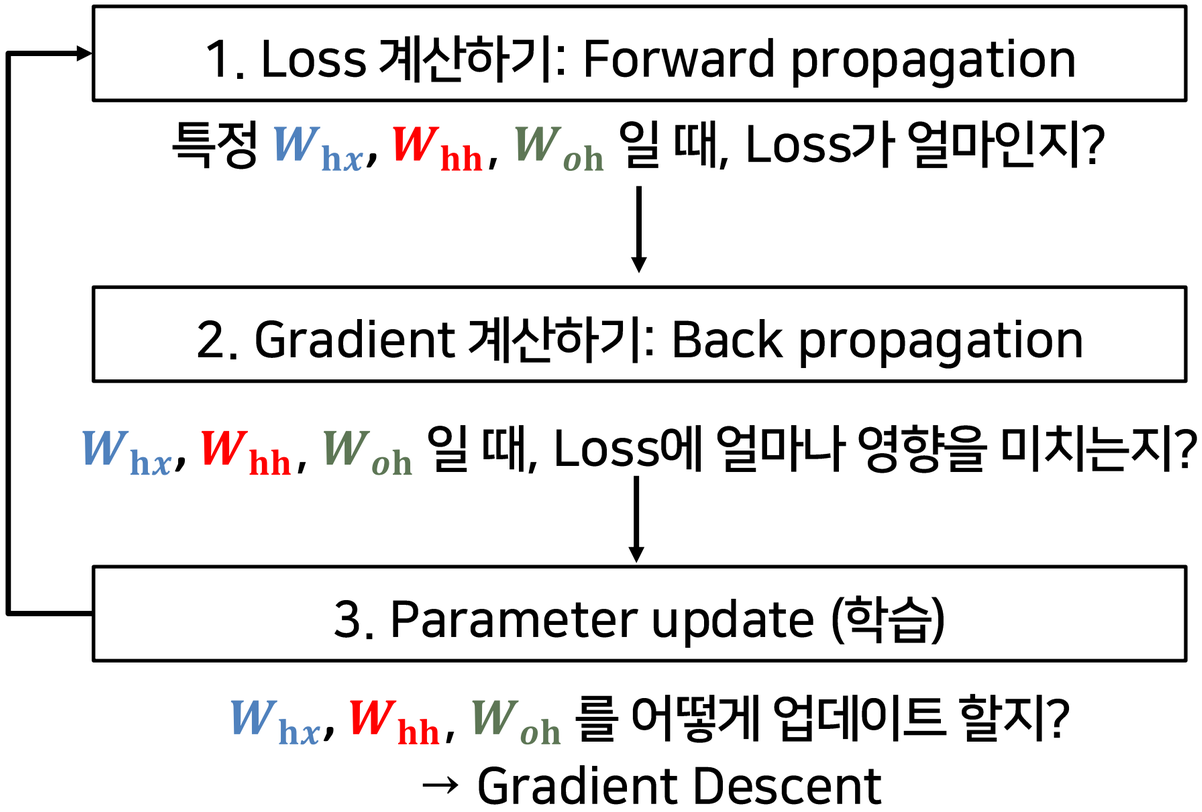




05. RNN & LSTM

RNN의 학습

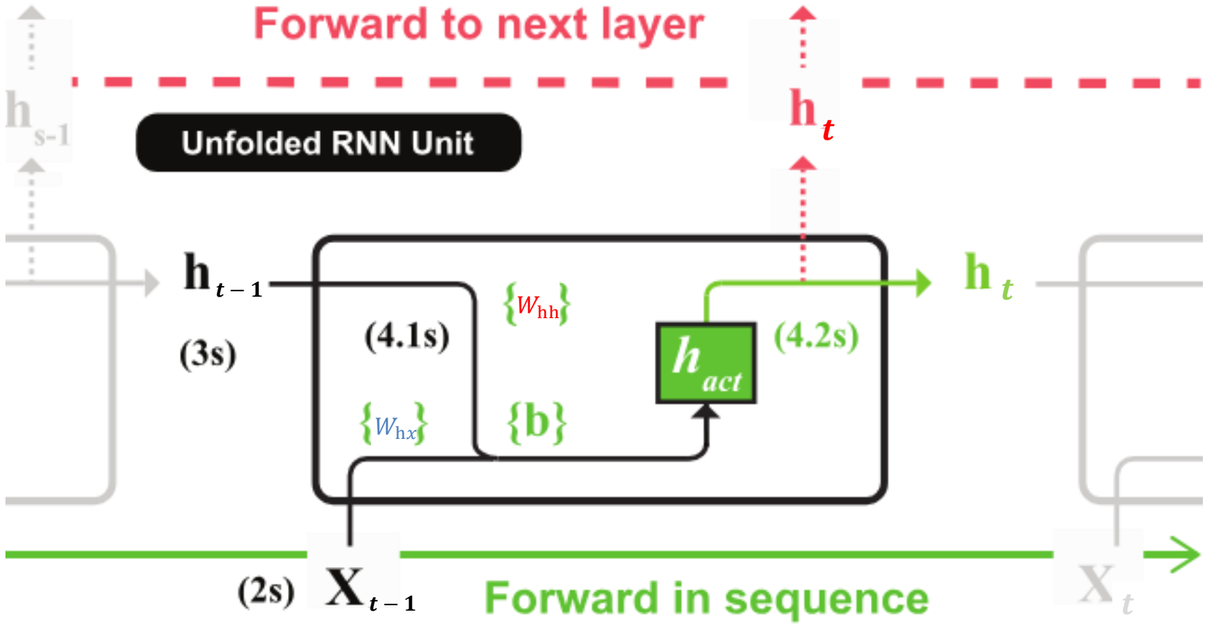
- 학습 대상 (parameters): (W_{hx}, W_{hh}, W_{oh})



05. RNN & LSTM

RNN의 학습 (Forward pass)

- $L_t = o_t$와 $\hat{o}_t$ 차이
- One to many, many to many 구조 같이 output이 여러 개인 구조에서는 각 시점 별 loss의 평균을 전체 loss 로 활용



$$\mathbf{h}_t = \tanh(W_{hx}x_t + W_{hh}\mathbf{h}_{t-1})$$

```
class RNN:
  def __init__(self, Wx, Wh, b):
    self.params = [Wx,Wh,b]
    self.grads = [np.zeros_like(Wx), np.zeros_like(Wh), np.zeros_like(b)]
    self.cache = None

  def forward(self, x, h_prev):
    Wx, Wh, b = self.params
    t = np.matmul(h_prev, Wh) + np.matmul(x, Wx) + b
    h_next = np.tanh(t)
```

$$\hat{o}_t = \mathbf{softmax}(W_{oh}\mathbf{h}_t)$$

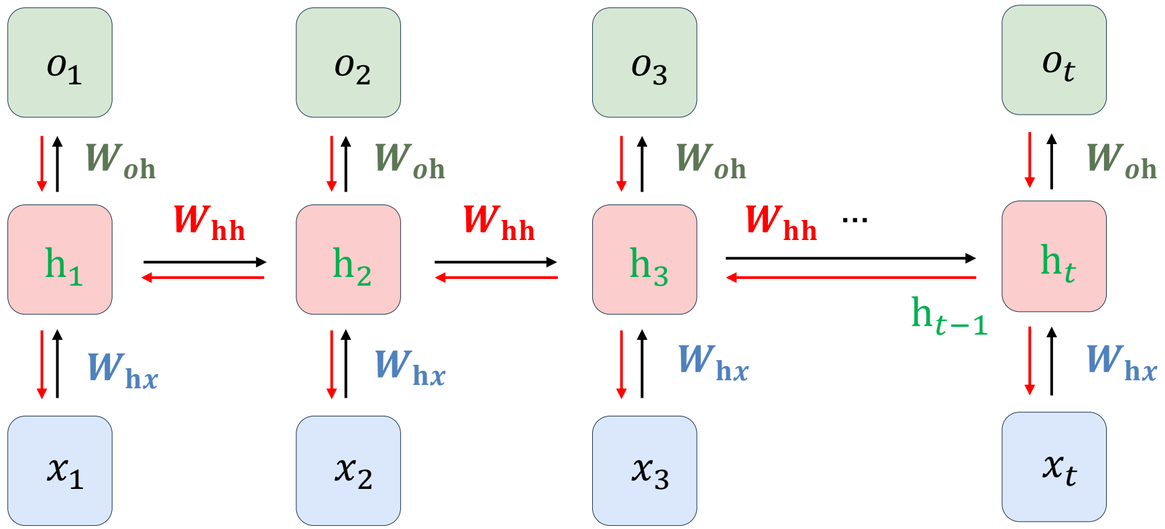
$$L(x, y, W_{hx}, W_{hh}, W_{oh}) = \sum_{t=1}^{T} l(o_t, \hat{o}_t)$$

05. RNN & LSTM

RNN의 학습 (Backpropagation)

- Backward Propagation Through Time (BPTT) in RNN

: Time step에 따라 Back Propagation

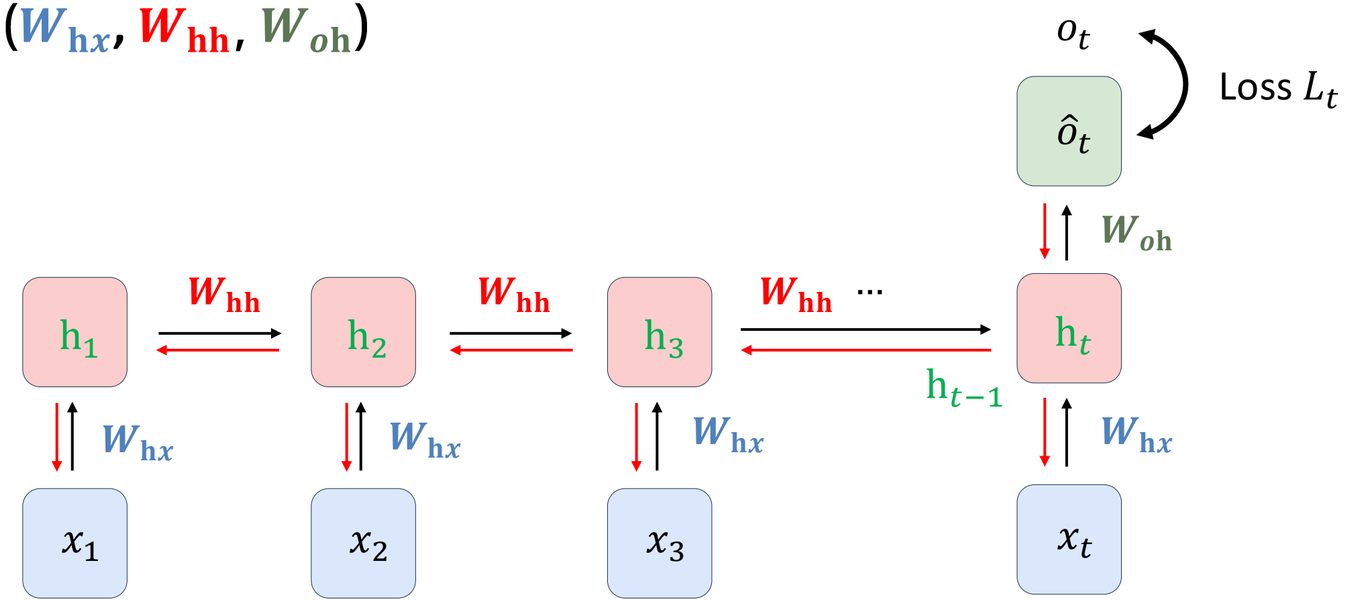


05. RNN & LSTM

RNN의 학습 (Backpropagation)

- Backward Propagation Through Time (BPTT) in RNN

: 학습 대상 (parameters): (W_{hx}, W_{hh}, W_{oh})



$$\frac{\partial Loss}{\partial W_{oh}} = \frac{\partial L_t}{\partial \hat{o}_t} \times \frac{\partial \hat{o}_t}{\partial W_{oh}\mathbf{h}_t} \times \frac{\partial W_{oh}\mathbf{h}_t}{\partial W_{oh}}$$

05. RNN & LSTM

RNN의 학습 (Backpropagation)

- Backward Propagation Through Time (BPTT) in RNN

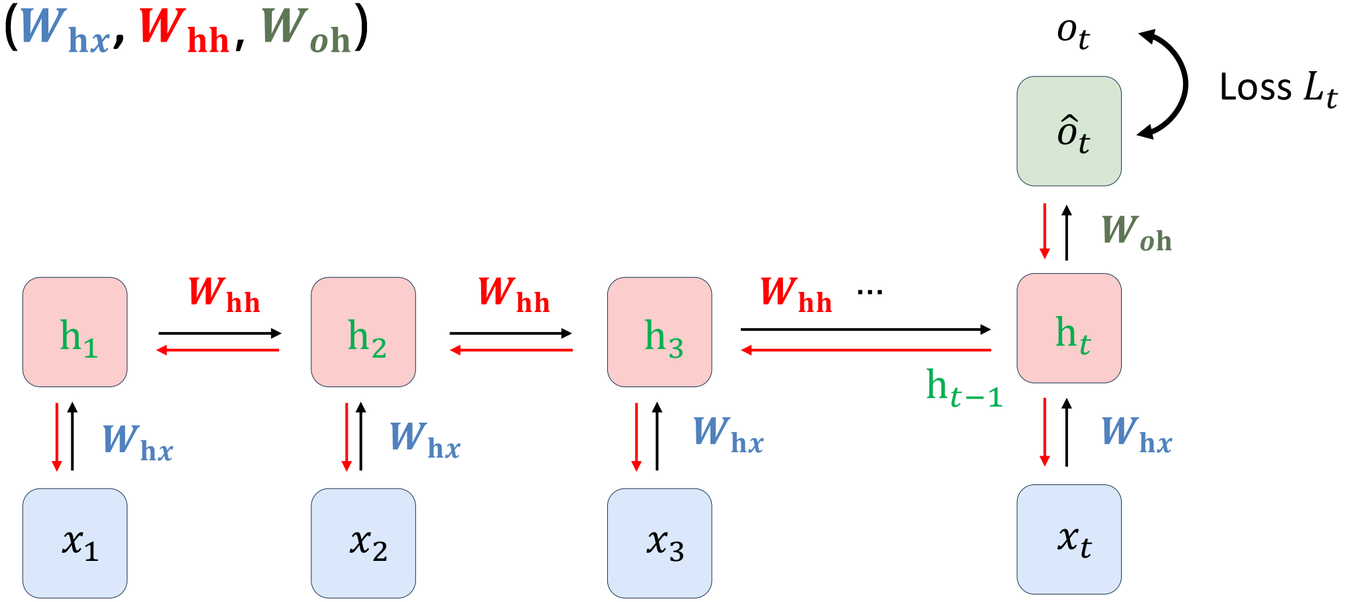
: 학습 대상 (parameters): (W_{hx}, W_{hh}, W_{oh})

(Forward)

$\mathbf{h}_t = \tanh(W_{hx}x_t + W_{hh}\mathbf{h}_{t-1})$

$\mathbf{h}_{t-1} = \tanh(W_{hx}x_{t-1} + W_{hh}\mathbf{h}_{t-2})$

…



$$\frac{\partial Loss}{\partial W_{hh}} = \frac{\partial L_t}{\partial \hat{o}_t} \times \frac{\partial \hat{o}_t}{\partial \mathbf{h}_t} \times \frac{\partial \mathbf{h}_t}{\partial W_{hh}} + \frac{\partial L_t}{\partial \hat{o}_t} \times \frac{\partial \hat{o}_t}{\partial \mathbf{h}_t} \times \frac{\partial \mathbf{h}_t}{\partial \mathbf{h}_{t-1}} \times \frac{\partial \mathbf{h}_{t-1}}{\partial W_{hh}} + \cdots$$

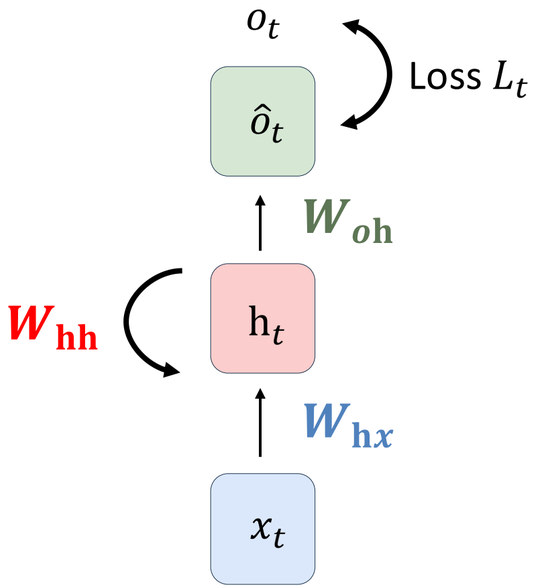
$$\frac{\partial Loss}{\partial W_{hx}} = \frac{\partial L_t}{\partial \hat{o}_t} \times \frac{\partial \hat{o}_t}{\partial \mathbf{h}_t} \times \frac{\partial \mathbf{h}_t}{\partial W_{hx}} + \frac{\partial L_t}{\partial \hat{o}_t} \times \frac{\partial \hat{o}_t}{\partial \mathbf{h}_t} \times \frac{\partial \mathbf{h}_t}{\partial \mathbf{h}_{t-1}} \times \frac{\partial \mathbf{h}_{t-1}}{\partial W_{hx}} + \cdots$$

05. RNN & LSTM

RNN의 학습 (Backpropagation)

- Backward Propagation Through Time (BPTT) in RNN

: 학습 대상 (parameters): (W_{hx}, W_{hh}, W_{oh})



$$\frac{\partial Loss}{\partial W_{oh}} = \frac{\partial L_t}{\partial \hat{o}_t} \times \frac{\partial \hat{o}_t}{\partial W_{oh}\mathbf{h}_t} \times \frac{\partial W_{oh}\mathbf{h}_t}{\partial W_{oh}}$$

$$\frac{\partial Loss}{\partial W_{hh}} = \sum_{k=0}^{t} \left(\frac{\partial L_t}{\partial \hat{o}_t} \times \frac{\partial \hat{o}_t}{\partial \mathbf{h}_t} \times \frac{\partial \mathbf{h}_t}{\partial \mathbf{h}_k} \times \frac{\partial \mathbf{h}_k}{\partial W_{hh}} \right)$$

$$\frac{\partial Loss}{\partial W_{hx}} = \sum_{k=0}^{t} \left(\frac{\partial L_t}{\partial \hat{o}_t} \times \frac{\partial \hat{o}_t}{\partial \mathbf{h}_t} \times \frac{\partial \mathbf{h}_t}{\partial \mathbf{h}_k} \times \frac{\partial \mathbf{h}_k}{\partial W_{hx}} \right)$$

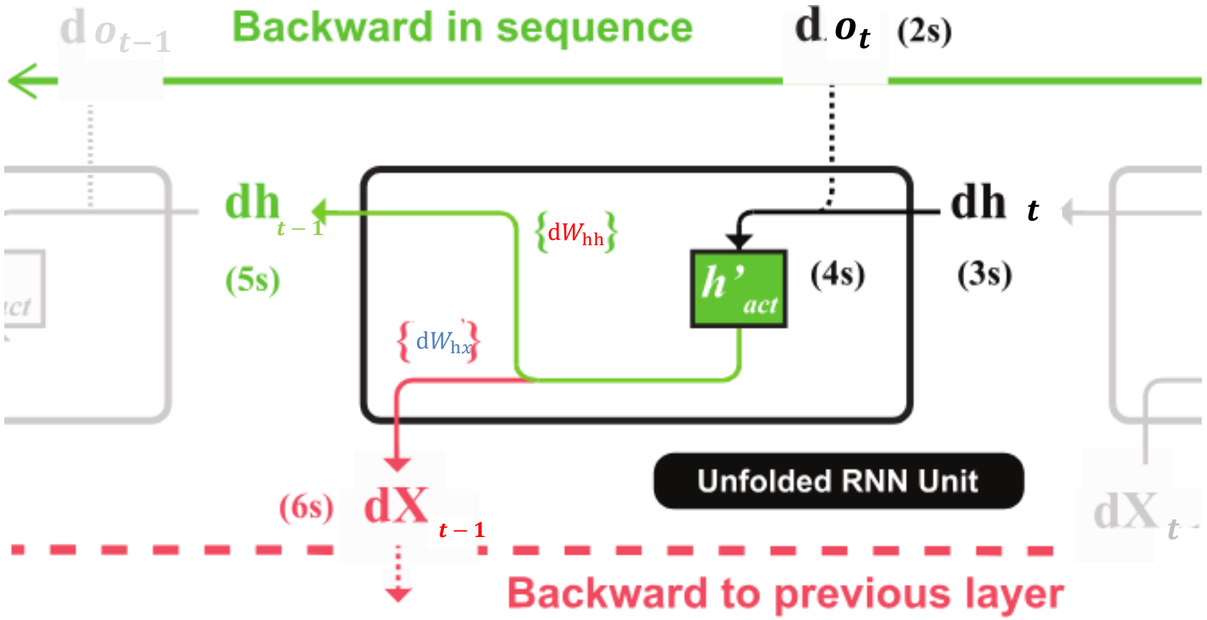
- 회귀 문제: Loss function (MSE)
- 분류 문제: Loss function (Cross Entropy)

05. RNN & LSTM

RNN의 학습 (Backpropagation)

- Backward Propagation Through Time (BPTT) in RNN

: Time step에 따라 Back Propagation

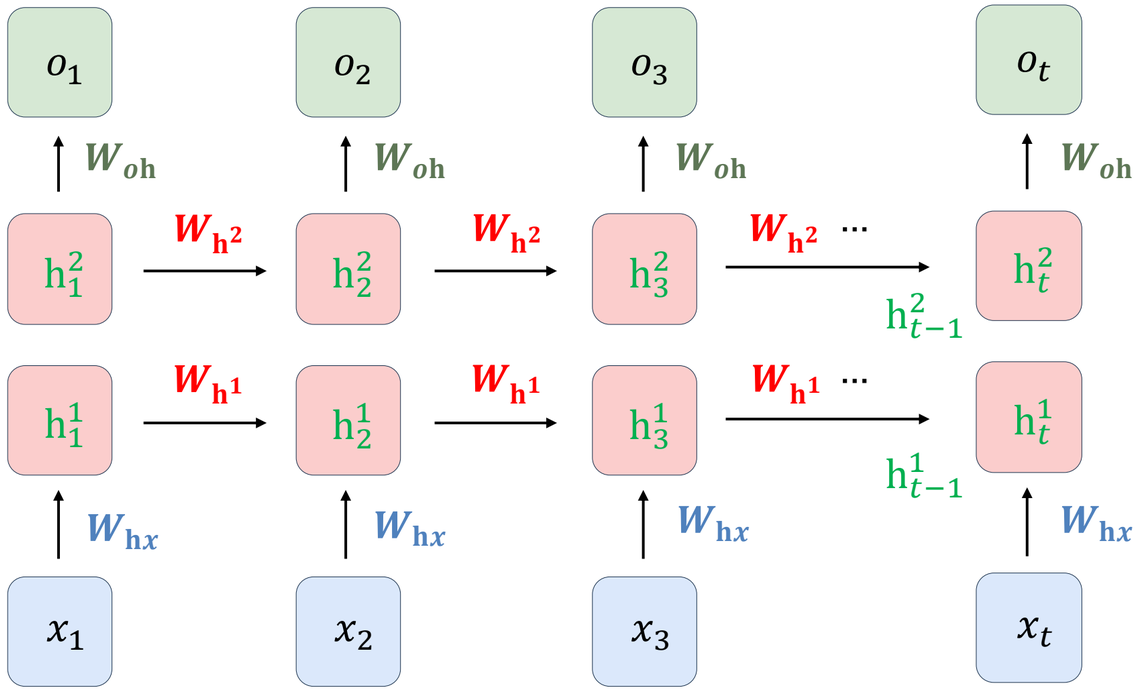



```python
def backward(self, dh_next):
  Wx, Wh, b = self. params
  x, h_prev, h_next = self.cache
  dt = dh_next * (1 - h_next ** 2)
  db = np. sum(dt, axis=0)
  dWh = np. matmul(h_prev.T, dt)
  dh_prev = np. matmul(dt, Wh.T)
  dWx = np.matmul(x.T, dt)
  dx = np. matmul(dt, Wx.T)
  self.grads[0][...] = dWx
  self.grads[1][...] = dWh
  self.grads[2][...] = db
  return dx, dh_prev
```

05. RNN & LSTM

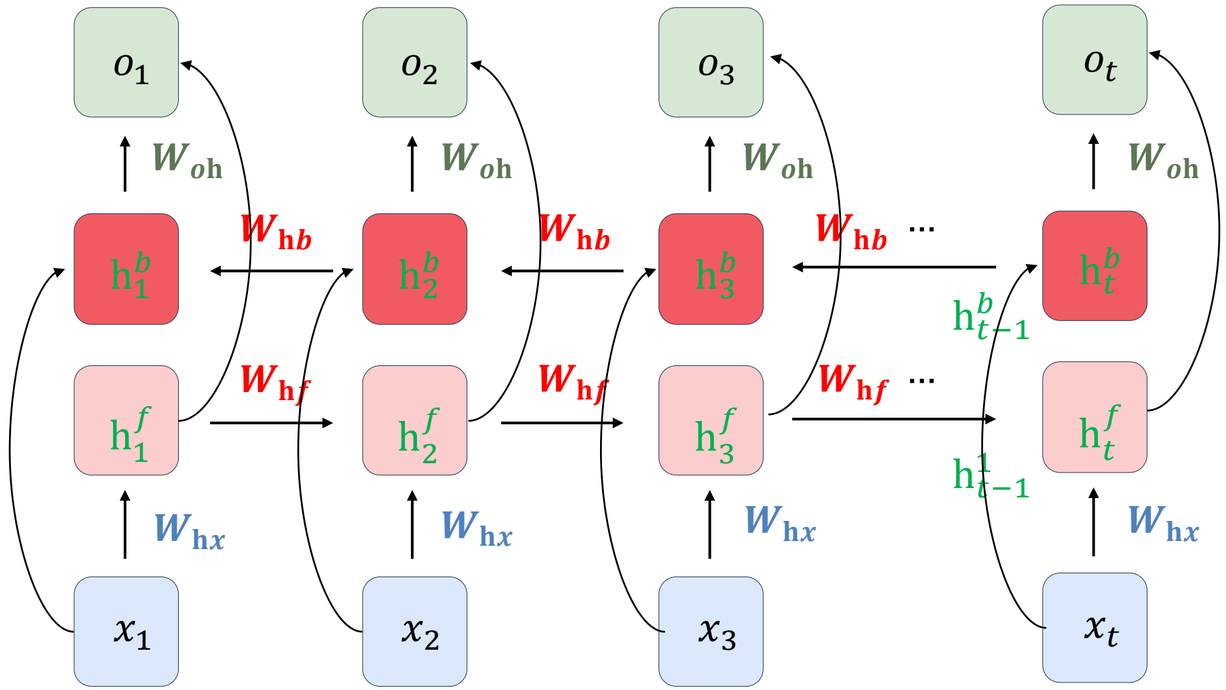
Deep, Bidirectional RNN

- Deep RNN



Hidden layer를 여러 개 쌓은 형태
코드 option: num_layers = c

- Bidirectional RNN



앞 시점의 hidden state과 뒤 시점의 hidden state을
사용하는 양방향식
코드 option: bidirectional = True

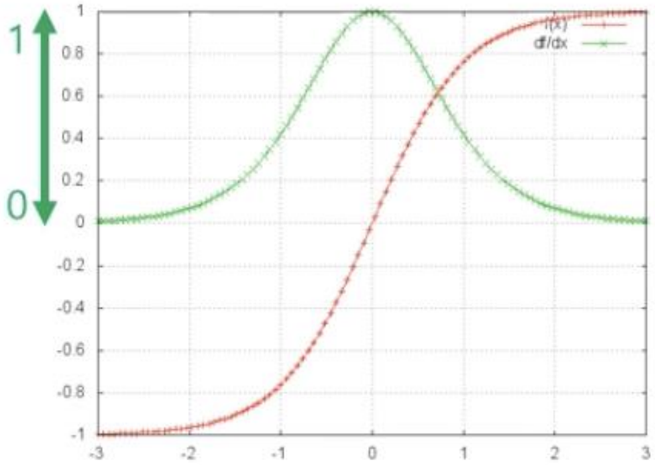
05. RNN & LSTM

RNN의 한계

- 장기의존성 문제 (Long-term dependency problem)
- Sequence의 길이가 길어질수록 과거 정보 학습에 어려움 발생

:원인: 기울기 소실 (gradient vanishing)

: ex. $$W_{\mathrm{hh}}{}^{new} = W_{\mathrm{hh}}{}^{old} - \eta * \left(\frac{\partial Loss}{\partial \hat{o}_t} \times \frac{\partial \hat{o}_t}{\partial \mathrm{h}_{100}} \times \frac{\partial \mathrm{h}_{100}}{\partial W_{\mathrm{hh}}} + \cdots + \frac{\partial Loss}{\partial \hat{o}_t} \times \frac{\partial \hat{o}_t}{\partial \mathrm{h}_{100}} \times \cdots \times \frac{\partial \mathrm{h}_3}{\partial \mathrm{h}_2} \times \frac{\partial \mathrm{h}_2}{\partial \mathrm{h}_1} \times \frac{\partial \mathrm{h}_1}{\partial W_{\mathrm{hh}}}\right)$$



$tanh$

$\frac{d}{dx} tanh$

$$\mathrm{h}_t = \tanh(W_{x\mathrm{h}} x_t + W_{\mathrm{hh}} \mathrm{h}_{t-1})$$

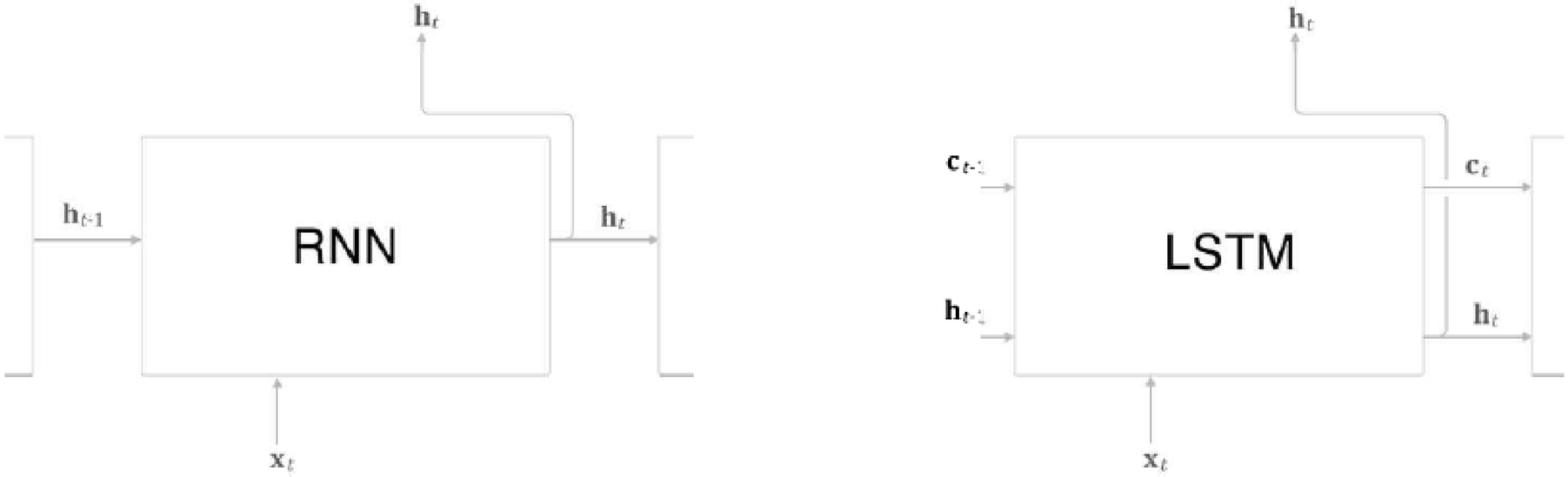
$$\frac{\partial \mathrm{h}_t}{\partial \mathrm{h}_{t-1}} = \underbrace{\tanh'(W_{x\mathrm{h}} x_t + W_{\mathrm{hh}} \mathrm{h}_{t-1}) W_{\mathrm{hh}}}_{\text{0부터 1 사이 값을 지님}}$$

05. RNN & LSTM

Long Short Term Memory (LSTM)

- Long Short Term Memory (LSTM)

: 기존 RNN architecture에 Gate을 추가함으로써 기억력을 개선하는 기법



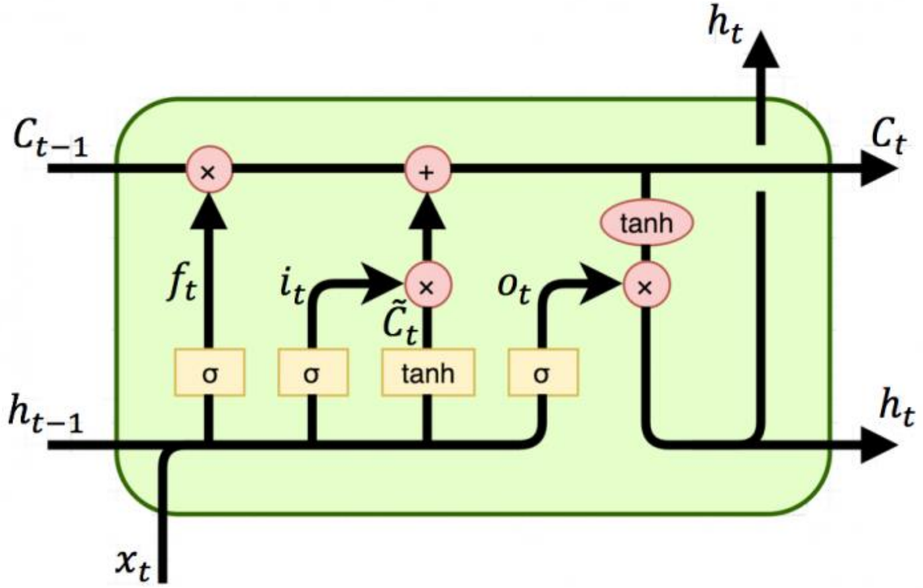
: Hidden state만 존재하던 RNN과 달리 cell state가 추가됨.
: c_t에는 t 시점까지의 기억 정보들이 누적 (=장기적으로 정보들을 유지)
: cell state에서 정보들을 얼마나 기억할지 gate을 통해 정할 수 있음.

05. RNN & LSTM

Long Short Term Memory (LSTM)

- LSTM architecture: three gates

Gate: Forget gate (f_t), Input gate (i_t) Output gate (o_t)



LSTM은 아래 세 가지 gate로 이전 정보를 얼마나 기억하고 잊을지를 정한다!

- Forget gate
- Input gate
- Output gate

] Cell state c_t 업데이트

$$f_t = \sigma\left(W_{xf}x_t + W_{hf}\mathbf{h}_{t-1} + b_f\right)$$

$$i_t = \sigma(W_{xi}x_t + W_{hi}\mathbf{h}_{t-1} + b_i)$$

$$o_t = \sigma(W_{xo}x_t + W_{ho}\mathbf{h}_{t-1} + b_o)$$

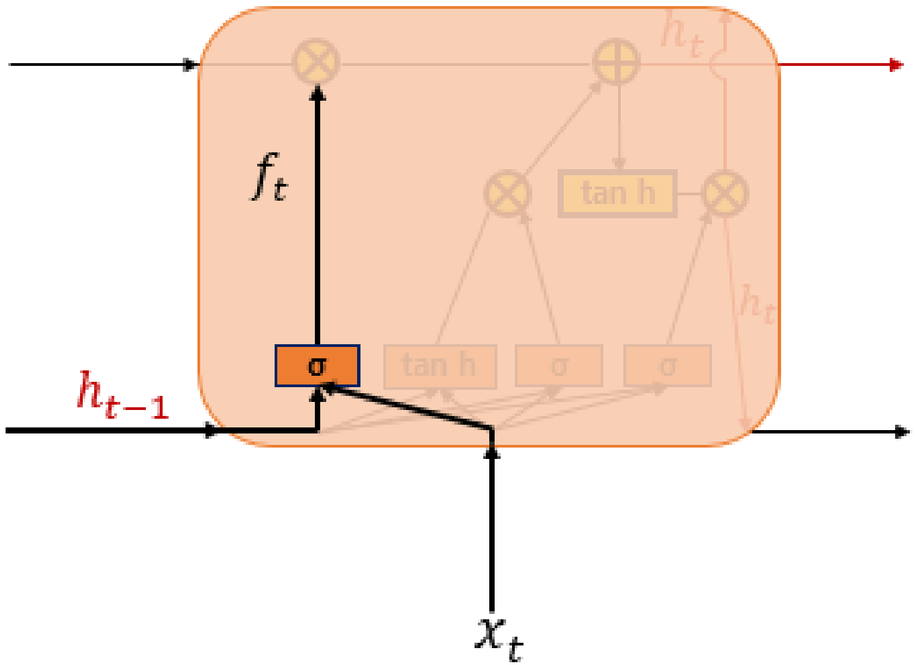
→ 세 게이트 모두 sigmoid를 통해 0과 1 사이 값 출력

05. RNN & LSTM

Long Short Term Memory (LSTM)

- LSTM three gates

: Forget gate (f_t)



이전 기억을 얼마나 삭제할 것인지 정하는 gate

$$f_t = \sigma\left(W_{xf}x_t + W_{hf}\mathbf{h}_{t-1} + b_f\right)$$

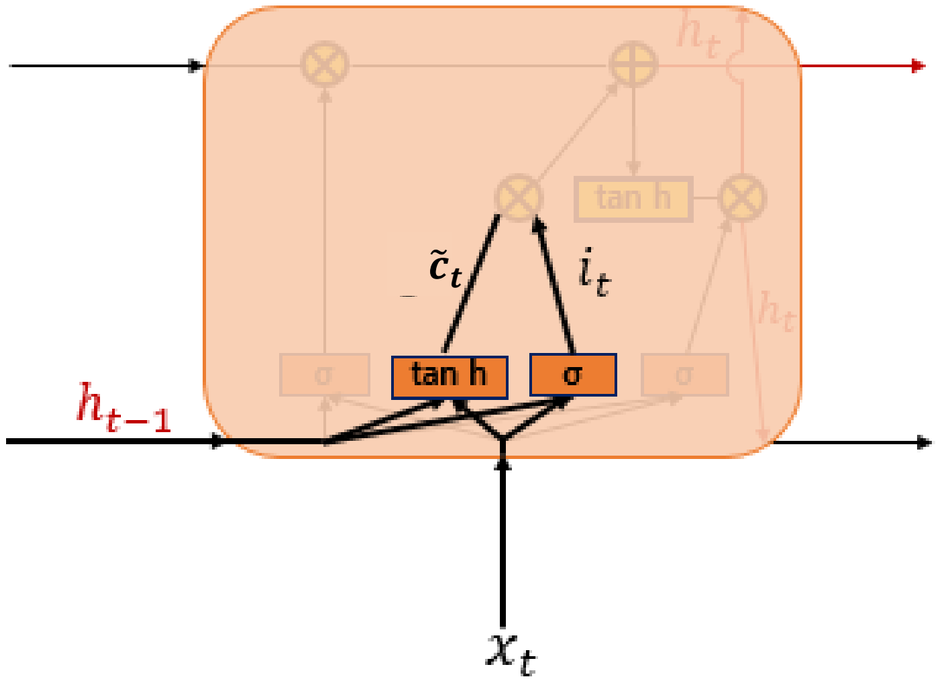
- f_t 는 직전 시점의 cell state(c_{t-1}) 와 곱해짐.
- 0에 가까울수록 이전 기억이 많이 삭제, 1에 가까울수록 이전 기억 보존.
- 얼마나 기억할 것인지를 가중치를 업데이트하며 학습.

05. RNN & LSTM

Long Short Term Memory (LSTM)

- LSTM three gates

: Input gate (i_t)



새로운 정보를 얼마나 누적할 것인지 정하는 gate

$$i_t = \sigma(W_{xi}x_t + W_{hi}\mathbf{h}_{t-1} + b_i) \quad \rightarrow 0\text{에서 } 1 \text{ 값 출력}$$

$$\tilde{c}_t = \tanh\left(W_{x\tilde{c}}x_t + W_{h\tilde{c}}\mathbf{h}_{t-1} + b_g\right) \quad \rightarrow -1\text{에서 } 1 \text{ 값 출력}$$

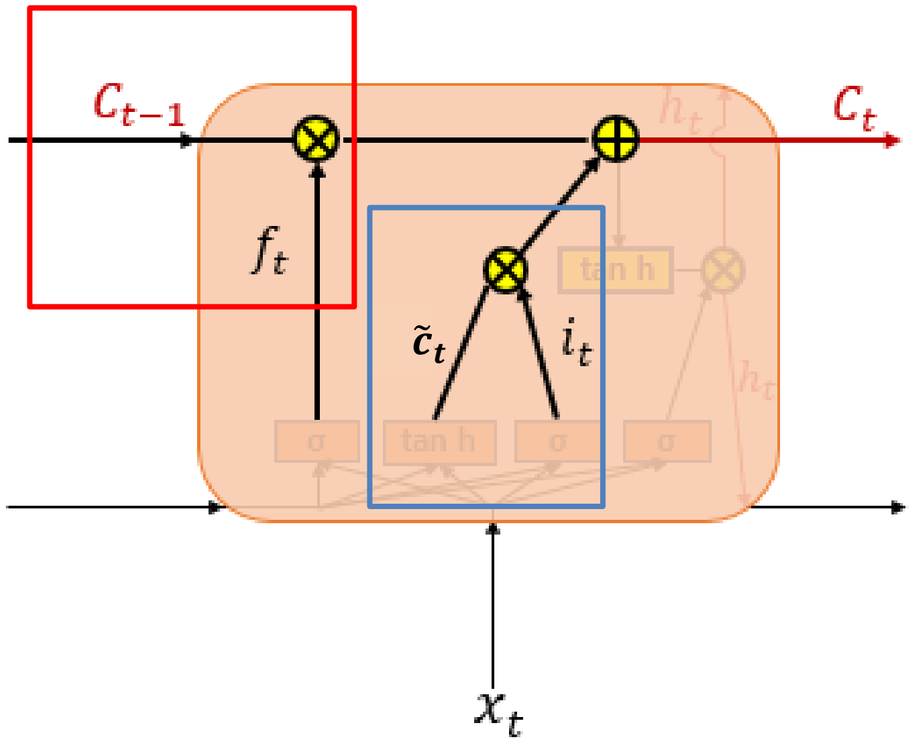
- $\tilde{c}_t$는 현재 입력 값과 과거 hidden state의 정보 요약 값. (=임시 cell state)
- i_t 는 $\tilde{c}_t$와 곱해지며 1에 가까울수록 새로운 정보를 온전히 누적.
- 얼마나 누적할 것인지는 가중치를 업데이트하며 학습.

05. RNN & LSTM

Long Short Term Memory (LSTM)

- LSTM three gates

: Cell state



$$c_t = f_t \otimes c_{t-1} \oplus i_t \otimes \tilde{c}_t$$

⊗ 는 원소별 곱 (elementwise product)!

f_t	0 | 0.2 | 0.9 | 0.1 | 1	0.1 | 0 | 0.8 | 0.2 | 0.8	i_t
C_{t-1}	-0.2 | 0.1 | 0.5 | 0.7 | 0.9	-0.1 | 0.3 | 0.6 | 0.2 | 0.9	$\tilde{c}_t$
$f_t \otimes C_{t-1}$	0 | 0.02 | 0.45 | 0.07 | 0.9 ⊕	-0.01 | 0 | 0.48 | 0.04 | 0.72	$i_t \otimes \tilde{c}_t$

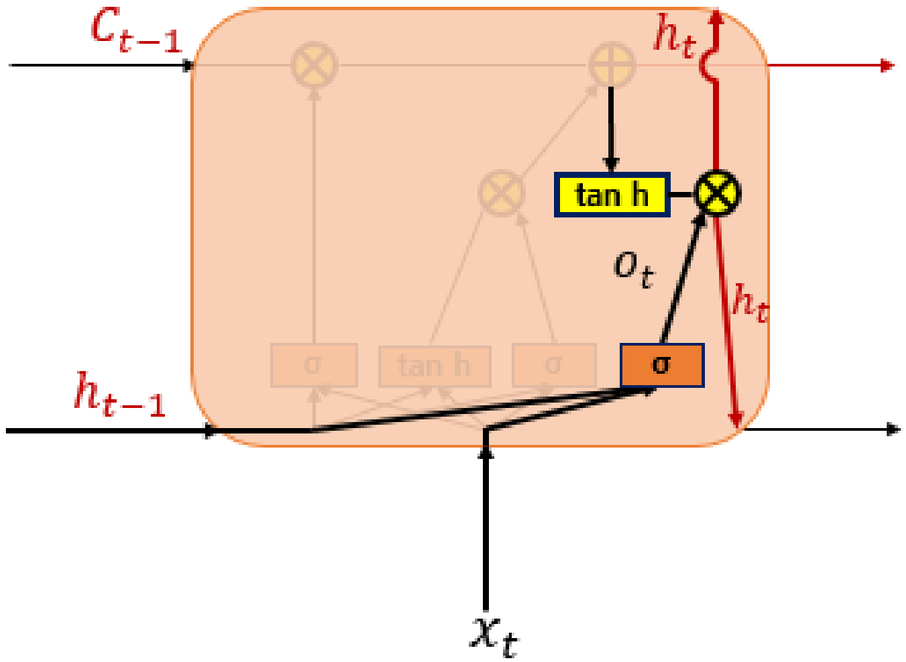
** Gradient Vanishing 방지 원리?

05. RNN & LSTM

Long Short Term Memory (LSTM)

- LSTM three gates

: Output gate (o_t)



Input, forget gate의 값들을 다음 단계로 전달해주는 gate

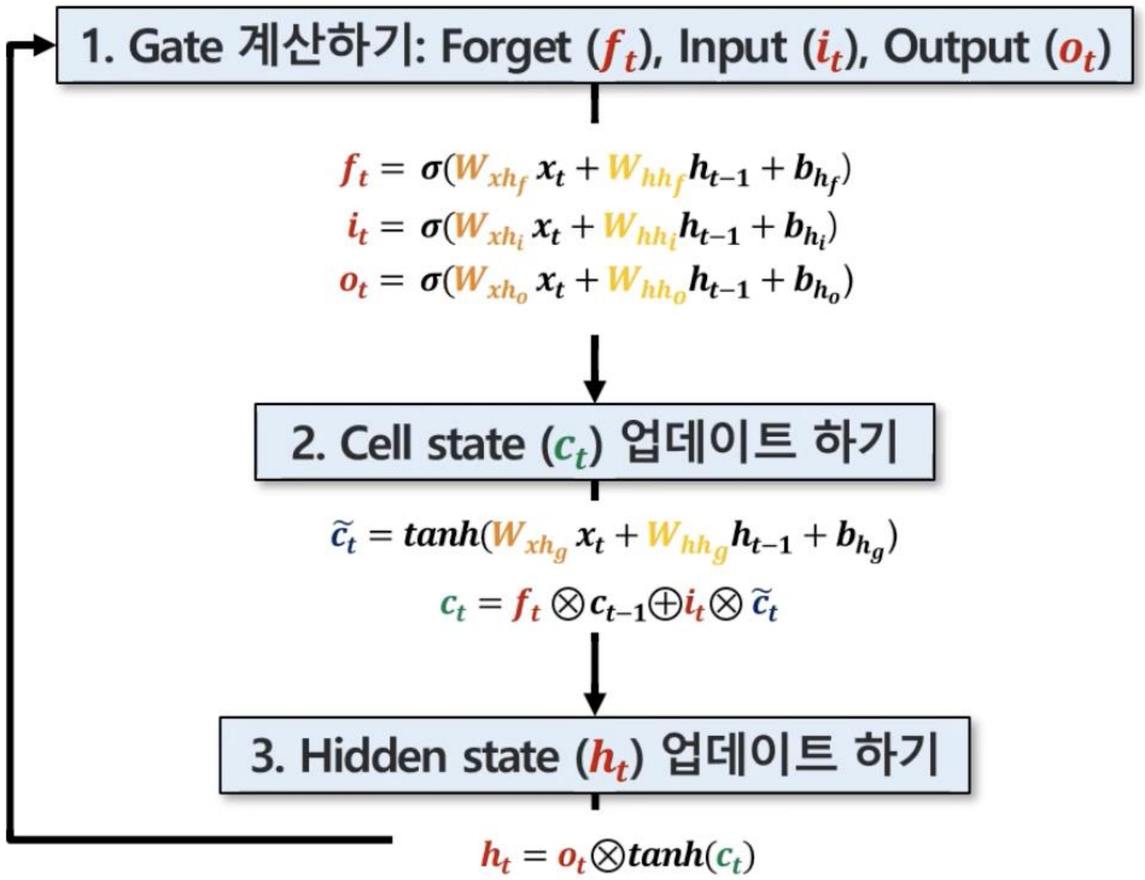
$o_t = \sigma(W_{xo}x_t + W_{ho}\mathbf{h}_{t-1} + b_o)$ → 0에서 1 값 출력

$\mathrm{h}_t = o_t \otimes \tanh(c_t)$

- o_t 는 $\tanh(c_t)$ 와 곱해져 hidden state h_t로 업데이트.
- 즉 o_t 를 통해 hidden state에 cell state를 얼마나 반영할 것인지 결정.
- 마찬가지로 원소별 곱 $\otimes$ 사용

05. RNN & LSTM

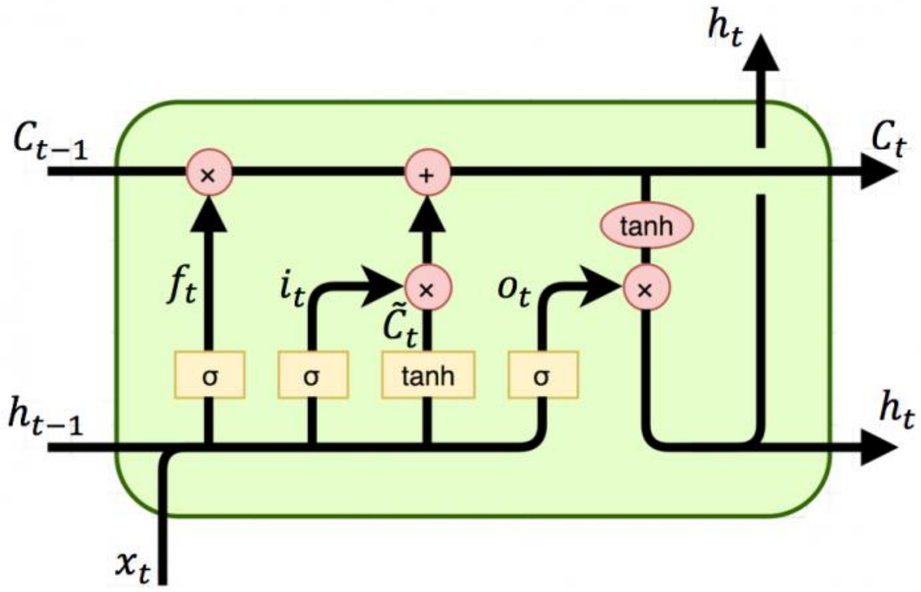
Long Short Term Memory (LSTM)



05. RNN & LSTM

Long Short Term Memory (LSTM)

Gate: Forget gate (f_t), Input gate (i_t) Output gate (o_t)



$$f_t = \sigma\left(W_{xf}x_t + W_{hf}\mathbf{h}_{t-1} + b_f\right)$$

$$i_t = \sigma(W_{xi}x_t + W_{hi}\mathbf{h}_{t-1} + b_i)$$

$$\tilde{c}_t = \tanh\left(W_{x\tilde{c}}x_t + W_{h\tilde{c}}\mathbf{h}_{t-1} + b_g\right)$$

cell state $c_t = f_t \otimes c_{t-1} \oplus i_t \otimes \tilde{c}_t$

hidden state $\mathrm{h}_t = o_t \otimes \tanh(c_t)$

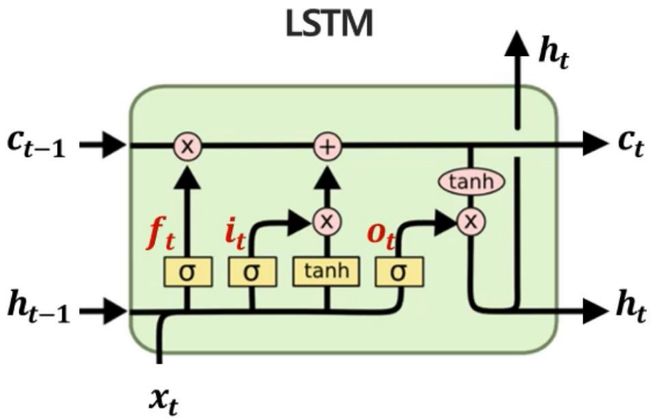
$$o_t = \sigma(W_{xo}x_t + W_{ho}\mathbf{h}_{t-1} + b_o)$$

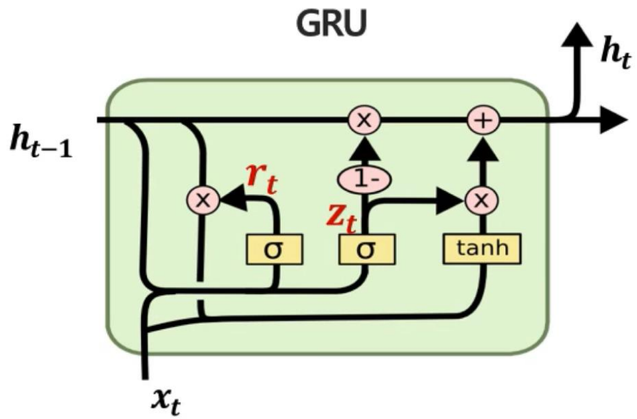
05. RNN & LSTM

Gated Recurrent Unit (GRU)

- Gated Recurrent Unit (GRU)

: LSTM에서 gate을 2개로 줄임으로써 성능은 유지하되 속도 및 연산을 개선시킨 모델





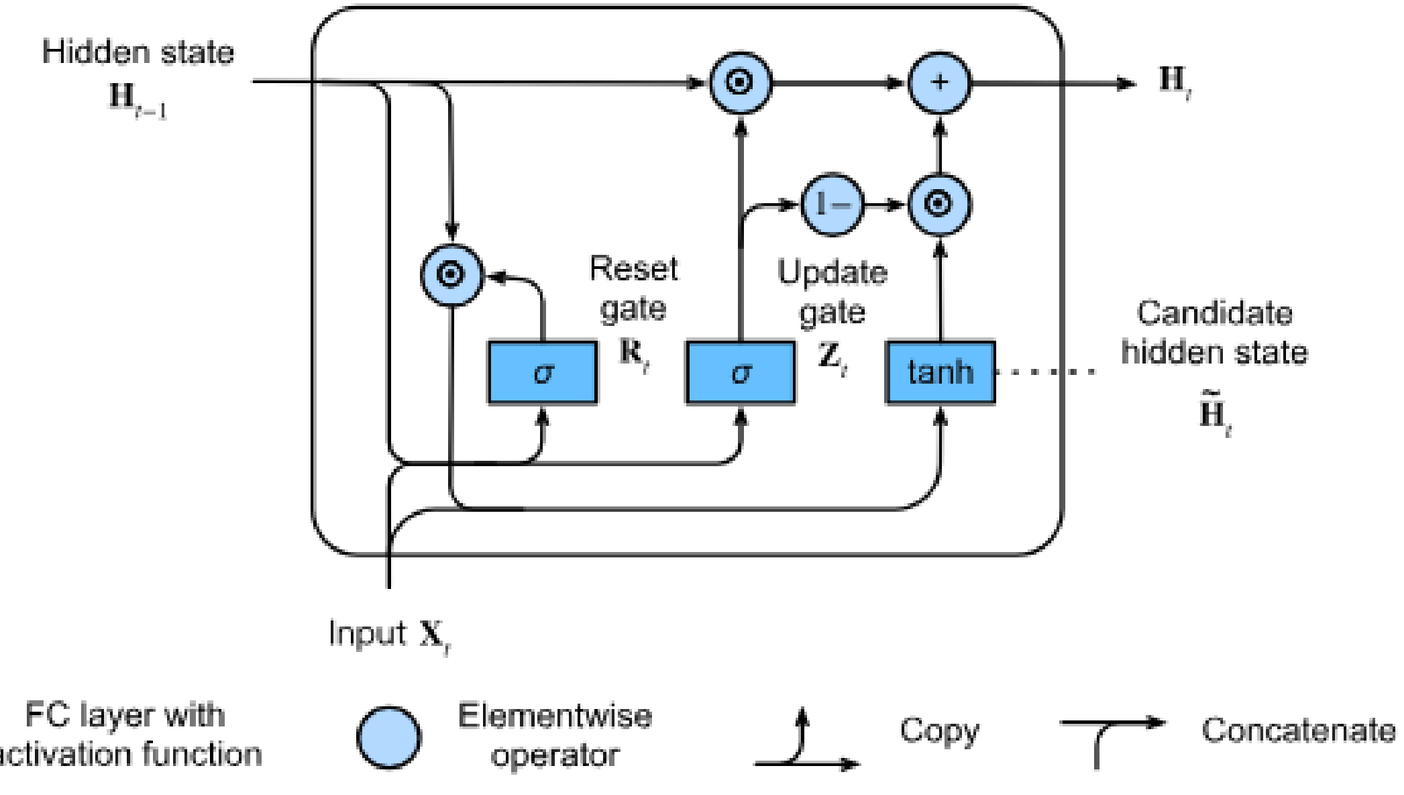
GRU는 LSTM의 구조를 간단히 개선하여 파라미터 수를 줄임

- Forget gate, input gate를 update gate(z_t)로 통합
- Output gate을 없애고 reset gate(r_t)를 정의
- Cell state, hidden state를 hidden state로 통합

05. RNN & LSTM

Gated Recurrent Unit (GRU)

- 참고) GRU 아키텍처

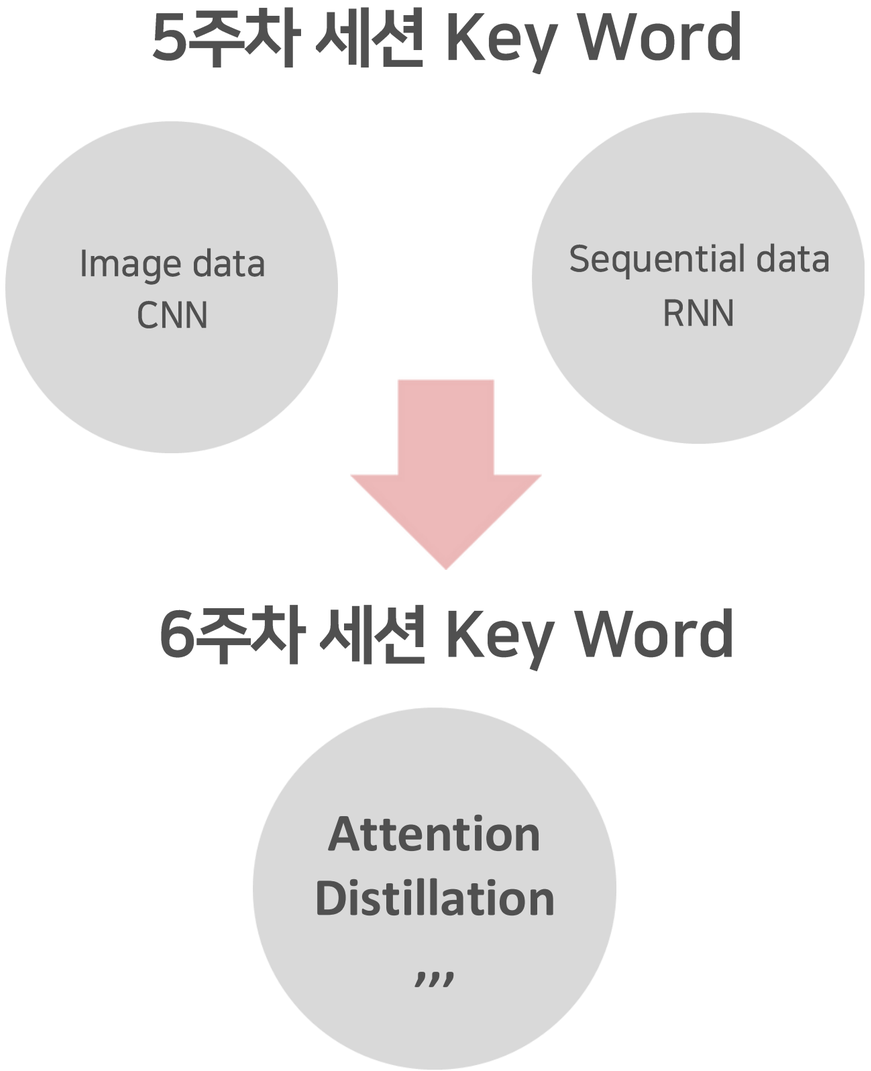


$$
\begin{aligned}
z_t &= \sigma(W_z x_t + U_z h_{t-1} + b_z) \\
r_t &= \sigma(W_r x_t + U_r h_{t-1} + b_r) \\
\hat{h}_t &= \phi(W_h x_t + U_h(r_t \odot h_{t-1}) + b_h) \\
h_t &= (1 - z_t) \odot h_{t-1} + z_t \odot \hat{h}_t
\end{aligned}
$$

06

다음 주차 예고

06. 다음 주차 예고



06. 다음 주차 예고

5주차 과제

공부 과제

d2l
9장, 10장
RNN, LSTM

논문 리뷰
ResNet

코드 과제

택1
1) CNN 구현
2) ResNet 논문만 읽고
핵심 모듈 구현해보기

Thank You

2026 Winter DL Session 5주차 끝!

고려대학교 KOREA UNIVERSITY

KUBIG DATA SCIENCE & AI